AI / ML FOUNDATIONS • STAGE 1 • COMPUTATIONAL LAYER

NumPy

Master Notes

Understanding Data Types in Python + The Basics of NumPy Arrays

Based on: Python Data Science Handbook, Chapter 2 (Blocks 1 and 2)

Theory | Examples | Visual Outputs | Practice Problems | Complete Answer Key

WHAT IS INSIDE

- 1. Why NumPy beats lists
- 2. Creating arrays
- 3. Data types
- 4. Array attributes
- 5. Indexing
- 6. Slicing
- 7. Views vs copies
- 8. Reshaping + newaxis
- 9. Concatenate + split

Slow-paced notes: read, understand, then practise. Answers are at the very end.

Prepared for Abhijeet • October 2026

How to use these notes

These notes cover the two blocks you studied from the Python Data Science Handbook: **Block 1: Understanding Data Types in Python** and **Block 2: The Basics of NumPy Arrays**. Every topic is explained in the same order: the idea first, then a real-life picture, then code, then the exact output you will see in Colab. After every topic you get practice problems. All correct answers are collected in the **Answer Key** at the very end, with the real output of every program.

Because you said you cannot memorise the syntax of every function, each function carries a priority tag so you know where to spend your energy:

Tag	Meaning	What to do
MUST KNOW	You will type this almost every day in AI/ML work.	Practise until your fingers know it.
GOOD TO KNOW	Used regularly, but not every day.	Know what it does and roughly how it looks.
LOOK UP	Rare, or easy to search when needed.	Remember that it exists. Do not memorise.

The coloured boxes you will meet:

ANALOGY	A real-life picture so the idea sticks.
MEMORY TRICK	A short trick to remember something forever.
INTERVIEW FLAG	The kind of question interviewers and viva examiners love to ask.
INTERVIEW POINT	Write or say this in an answer for full marks.
IMPORTANT	Pay attention: easy to get wrong.
COMMON TRAP	A mistake almost every beginner makes. NumPy never warns you about most of these.
TRY IT	Type this into Colab and watch it happen yourself.
WHY IT MATTERS	Where this idea shows up in real AI/ML work.
VERSION NOTE	Newer NumPy differs from the book in a small way.
NOTE	Extra detail that helps understanding.

The practice problem tags:

Tag	What you must do
PREDICT THE OUTPUT	Read the code and say exactly what Python will print. Do it in your head first, then run it.
FIND THE BUGS	The code looks fine but is broken or gives a wrong result. Find every mistake.
BUILD IT	Write the code yourself from a description.
THINK IT THROUGH	Reason about why. No memorising possible.
CALCULATE	Work out sizes and numbers step by step.
READ & DECODE	Read a line of NumPy and explain it in plain words.
THEORY	Interview-style written answers.

STUDY RULE

STUDY RULE. Do not read these notes like a story book only. Open Colab, type the examples, change one value at a time and watch what changes. NumPy is learnt with the fingers, not only the eyes.

All outputs in these notes were produced by running the code with NumPy 2.x, so what you see on the page is what you should see on your screen (except random numbers when you skip the seed).

Your NumPy topic map

#	Topic	Why it matters in AI/ML	Priority
1	Why NumPy beats lists	Explains vectorization, the idea behind all fast ML code	**MUST KNOW**
2	Creating arrays	zeros, arange, linspace, random.normal, eye: daily tools	**MUST KNOW**
3	Data types	float32 vs float64, uint8 for images, overflow bugs	**MUST KNOW**
4	Array attributes	Checking .shape prevents the most common ML bug	**MUST KNOW**
5	Indexing	Reading and writing single values	**MUST KNOW**
6	Slicing	Rows, columns and sub-tables of a dataset	**MUST KNOW**
7	Views vs copies	Silent data corruption if you forget .copy()	**MUST KNOW**
8	Reshaping + newaxis	Row/column vectors, preparing data for models	**MUST KNOW**
9	Concatenate + split	Adding features, building train/test splits	**MUST KNOW**

Contents

Part 1: Why NumPy? Lists vs Arrays . . . 6

1.1 Everything in Python is an object . . . 6

1.2 A Python list is more than just a list . . . 6

1.3 Vectorization: the real reason NumPy exists . . . 8

1.4 Fixed-type arrays in plain Python: the array module . . . 9

1.5 The standard import . . . 9

Part 2: Creating Arrays . . . 11

2.1 Creating an array from a Python list . . . 11

2.2 Upcasting: one type for the whole array . . . 11

2.3 Choosing the type yourself: the dtype keyword . . . 12

2.4 Multi-dimensional arrays: a list of lists . . . 12

2.5 Creating arrays from scratch . . . 13

Part 3: NumPy Data Types . . . 20

3.1 Why types matter so much in NumPy . . . 20

3.2 The standard NumPy types (from the book) . . . 20

3.3 Setting the dtype: two ways . . . 21

3.4 Checking the limits of a type . . . 22

3.5 Overflow: the silent bug . . . 22

3.6 The dtype controls memory . . . 23

3.7 Changing the type of an existing array: astype . . . 23

Part 4: Array Attributes . . . 25

4.1 What is an attribute? . . . 25

4.2 shape, ndim, size and dtype . . . 26

4.3 itemsize and nbytes . . . 26

4.4 How to read a shape tuple . . . 26

Part 5: Indexing . . . 29

5.1 Indexing a 1D array . . . 29

5.2 Indexing a 2D array: [row, column] . . . 30

5.3 Changing values, and the truncation trap . . . 30

Part 6: Slicing . . . 32

6.1 The slicing pattern . . . 32

6.2 Slicing 2D arrays . . . 33

6.3 Getting one row or one column . . . 33
6.4 Slicing never crashes on range . . . 34

Part 7: Views vs Copies . . . 36

7.1 Slices are views . . . 36
7.2 Making a real copy: .copy() . . . 37

Part 8: Reshaping and newaxis . . . 39

8.1 reshape . . . 39
8.2 Row vectors, column vectors and np.newaxis . . . 40

Part 9: Concatenation and Splitting . . . 43

9.1 np.concatenate: joining . . . 43
9.2 vstack, hstack and dstack . . . 44
9.3 np.split, vsplit and hsplit . . . 45

Part 10: Revision Zone . . . 47

10.1 Function cheat sheet . . . 47
10.2 Topic checklist . . . 48
10.3 Interview quick-fire: 20 questions with model answers . . . 48
10.4 The error radar: check these before you trust your output . . . 49

ANSWER KEY . . . 51

Answers: Part 1: Why NumPy? . . . 51
Answers: Part 2: Creating Arrays . . . 52
Answers: Part 3: Data Types . . . 53
Answers: Part 4: Array Attributes . . . 55
Answers: Part 5: Indexing . . . 56
Answers: Part 6: Slicing . . . 57
Answers: Part 7: Views vs Copies . . . 58
Answers: Part 8: Reshaping and newaxis . . . 59
Answers: Part 9: Concatenation and Splitting . . . 61
Answers: Final Mixed Paper (Blocks 1 and 2) . . . 62

PART 1

1 Why NumPy? Lists vs Arrays

Block 1, first half: Understanding Data Types in Python

IN THIS PART YOU WILL LEARN

- ✓ Why even a single number in Python is a heavy object
- ✓ What a Python list really stores, and why that costs memory and speed
- ✓ What vectorization means and why it is the reason NumPy exists
- ✓ Why the built-in **array** module is not enough, and what **ndarray** adds

1.1 Everything in Python is an object

In a language like C, an integer is just a few bytes of memory holding a number. In Python it is much more. When you write `x = 4`, Python builds a complete **object** that carries extra information about the number: its **type**, a **reference count** (how many names point to it), and then the **value** itself.

PYTHON • type this in Colab

```
import sys
x = 4
print(type(x))
print(sys.getsizeof(x), "bytes")
```

▶ OUTPUT

```
<class 'int'>
28 bytes
```

The number 4 needs one byte of real information, yet Python spends **28 bytes** on it. That overhead buys flexibility: you can write `x = 4` and a moment later `x = "four"` and Python does not mind. This is called **dynamic typing**.

NOTE Dynamic typing is great for everyday programming. It becomes a burden when you must store *millions* of numbers, because every one of them carries this overhead.

1.2 A Python list is more than just a list

The standard container for many items is the **list**. Because every item is a full object, a list can hold anything, even a mixture of types:

PYTHON • type this in Colab

```
L = list(range(10))
print(L)
print(type(L[0]))
L2 = [str(c) for c in L]
print(L2)
print(type(L2[0]))
L3 = [True, "2", 3.0, 4]
print([type(item) for item in L3])
```

▶ OUTPUT

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
<class 'int'>
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
<class 'str'>
[<class 'bool'>, <class 'str'>, <class 'float'>, <class 'int'>]
```

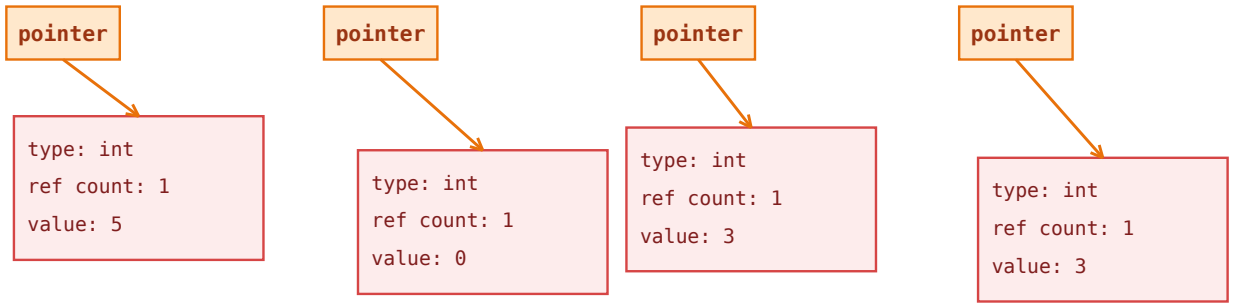
The price of this flexibility: **each item in the list contains its own type info, reference count and other data**. In the special case where every item has the same type, almost all of that information is repeated and wasted. A fixed-type array stores it just once.

ANALOGY

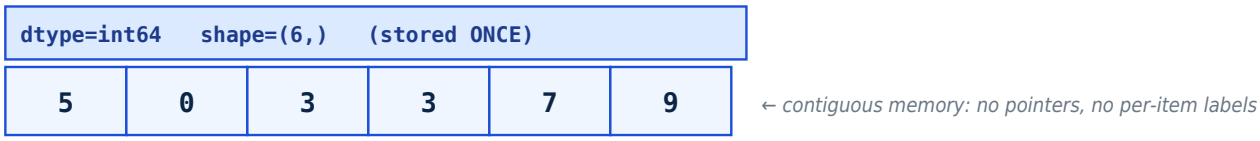
Python list = a shelf of gift boxes. Every box has its own label, its own instruction manual and the gift inside. The shelf itself only keeps a note saying where each box is. To read the 500th gift you must follow the note, find the box and open it.

NumPy array = an egg tray. Every slot holds the same kind of item, packed side by side, and the whole tray has *one* label. Reading the 500th egg is just counting 500 slots along.

PYTHON LIST: a row of pointers, each pointing to a full Python object



NUMPY ARRAY: one label for the whole block, values packed side by side



At the implementation level, a Python list holds a pointer to a block of pointers, and each of those points to a full Python object. A NumPy array holds a single pointer to **one contiguous block of memory**. Contiguous means "touching each other, with no gaps", just like the slots of the egg tray.

Point	Python list	NumPy array
What is stored	Pointers to full Python objects	Raw values packed in one block
Types allowed	Mixed types (flexible)	One fixed type only (strict)
Type information	Repeated inside every item	Stored once for the whole array
Memory used	Heavy	Compact
Speed of maths	Slow (Python loop, pointer chasing)	Fast (compiled C loop over one block)
Dimensions	Lists of lists (clumsy)	Real 1D, 2D, 3D... arrays

Seeing the memory difference with your own eyes

PYTHON • type this in Colab

```
import sys
L = list(range(1_000_000))
A = np.arange(1_000_000)
list_bytes = sys.getsizeof(L) + sum(sys.getsizeof(i) for i in L)
print("List  :", list_bytes, "bytes (approx.)")
print("Array :", A.nbytes, "bytes")
```

▶ OUTPUT

```
List  : 36000056 bytes (approx.)
Array : 8000000 bytes
```

The list needs about 4 to 5 times more memory for exactly the same one million numbers. The figure for the list counts the pointer table plus every integer object, so treat it as an estimate.

INTERVIEW FLAG

"Why is a NumPy array faster and smaller than a Python list?"

Model answer: a list stores pointers to full Python objects, each with its own type and reference count. A NumPy array stores raw values of one single type in one contiguous block, so it wastes no memory on per-item information and its operations run as tight C loops.

1.3 Vectorization: the real reason NumPy exists

Because an array knows that every element has the same type, NumPy can apply one operation to the **whole block at once** without you writing a loop. This is called **vectorization**. Look at what happens when we use * on a list and on an array:

PYTHON • type this in Colab

```
L = [1, 2, 3]
A = np.array(L)
print(L * 2)
print(A * 2)
print([x * 2 for x in L])
```

▶ OUTPUT

```
[1, 2, 3, 1, 2, 3]
[2 4 6]
[2, 4, 6]
```

COMMON TRAP

The same symbol means different things.

On a list, * means **repeat** and + means **join**. On a NumPy array both work **element by element**. If you want to double every value of a list you need a loop or a list comprehension, which is much slower on big data.

PYTHON • type this in Colab

```
print([1, 2] + [3, 4])
print(np.array([1, 2]) + np.array([3, 4]))
```

▶ OUTPUT

```
[1, 2, 3, 4]
[4 6]
```

WHY IT MATTERS

An ML dataset is millions of numbers of the same type. Training loops that would take hours in plain Python take seconds when written as vectorized NumPy (or PyTorch, which copies the same idea). **Rule for life: avoid Python loops over data, use vectorized operations.**

TRY IT

Measure it yourself in Colab. Run each line and compare the times:

`%timeit [x * 2 for x in range(1_000_000)]` versus `%timeit np.arange(1_000_000) * 2`

NumPy is typically many times faster. The exact figure depends on the machine, so we do not print a number here.

1.4 Fixed-type arrays in plain Python: the array module

Python does ship with a way to store fixed-type data compactly: the built-in `array` module.

PYTHON • type this in Colab

```
import array
L = list(range(10))
A = array.array('i', L)
A
```

▶ OUTPUT

```
array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

The `'i'` is a **type code** meaning "integer". The storage is compact, just like a NumPy array. So why does nobody use it for data science? Try the same doubling trick:

PYTHON • type this in Colab

```
B = array.array('i', [1, 2, 3])
print(B * 2)
```

▶ OUTPUT

```
array('i', [1, 2, 3, 1, 2, 3])
```

It *repeats* the array, exactly like a list. The module only gives you efficient **storage**. NumPy gives you efficient storage **and** efficient operations on it.

Feature	array.array	np.ndarray
Compact fixed-type storage	Yes	Yes
Element-wise maths (A * 2)	No (it repeats)	Yes
Multi-dimensional	No, 1D only	Yes, any number of dimensions
Used in ML work	Almost never	Everywhere

COMMON TRAP A frequent wrong belief: "the array module already does fast maths." It does **not**. It is only a compact container. Test `B * 2` once and you will never forget it.

1.5 The standard import

PYTHON • type this in Colab

```
import numpy as np
```

Everyone in the world writes `import numpy as np`. The short name `np` is a universal convention, so code you copy from tutorials and papers works without changes. The array object itself is called `ndarray` (n-dimensional array).

INTERVIEW FLAG **"What does ndarray stand for?"** N-dimensional array: one object that can be 1D, 2D, 3D or more.

✎ PRACTICE PROBLEMS | Part 1: Why NumPy?

Q1 THEORY

[3 marks] Give two reasons why a Python list is heavier than a NumPy array, and explain what "contiguous" means.

Q2 PREDICT THE OUTPUT

What does this print? Say what each item's type is.

PYTHON • type this in Colab

```
L3 = [True, "2", 3.0, 4]
print([type(item).__name__ for item in L3])
```

Q3 PREDICT THE OUTPUT

Predict both lines, then explain the difference in one sentence.

PYTHON • type this in Colab

```
L = [1, 2, 3]
A = np.array(L)
print(L * 3)
print(A * 3)
```

Q4 PREDICT THE OUTPUT

What do these two lines print?

PYTHON • type this in Colab

```
print([1, 2] + [3, 4])
print(np.array([1, 2]) + np.array([3, 4]))
```

Q5 THINK IT THROUGH

You must multiply **5 million** decimal sensor readings by 2. Do you store them in a list or a NumPy array? Give **two reasons**, each with a "because".

Q6 THINK IT THROUGH

A friend says: "Python already has fixed-type arrays in the array module, so why does anyone need NumPy?" Answer in 2 to 3 lines. Then prove it with one line of code.

Q7 CALCULATE

A NumPy array holds 1,000,000 values of type **int64** (8 bytes each). (a) How many bytes does it use? (b) How many if the type is **float32** (4 bytes)? Check with **.nbytes**.

PART 2

2 Creating Arrays

Block 1, second half: np.array, upcasting, dtype and every "from scratch" function

IN THIS PART YOU WILL LEARN

- ✓ How to turn Python lists into arrays, and what upcasting does to mixed types
- ✓ The **dtype** keyword: choosing the type yourself
- ✓ Every function that builds an array from scratch: zeros, ones, full, arange, linspace, random, eye, empty
- ✓ The difference between **arange** and **linspace** (the most confused pair)
- ✓ How **seed** makes random numbers repeatable

2.1 Creating an array from a Python list

The most direct way to build an array is `np.array`, which converts a list (or a list of lists) into an **ndarray**. **MUST KNOW**

PYTHON • type this in Colab

```
a = np.array([1, 4, 2, 5, 3])
a
```

▶ OUTPUT

```
array([1, 4, 2, 5, 3])
```

PYTHON • type this in Colab

```
print(type(a))
```

▶ OUTPUT

```
<class 'numpy.ndarray'>
```

NOTE

The result prints as `array([...])`, which is NumPy's way of saying "this is an ndarray". When you print it with `print()` the word *array* and the commas disappear: `[1 4 2 5 3]`.

2.2 Upcasting: one type for the whole array

A NumPy array can hold only **one** type. If you give it a mixture, NumPy quietly converts everything to the "biggest" type that can hold all the values. This is called **upcasting**.

PYTHON • type this in Colab

```
np.array([3.14, 4, 2, 3])
```

▶ OUTPUT

```
array([3.14, 4.  , 2.  , 3.  ])
```

The integers 4, 2 and 3 became floats (4., 2., 3.) so that the whole array is float. The order of strength is:

bool < int < float < string

PYTHON • type this in Colab

```
np.array([True, 5, 2.5])
```

▶ OUTPUT

```
array([1. , 5. , 2.5])
```

PYTHON • type this in Colab

```
np.array([1, 'a', 2.5])
```

▶ OUTPUT

```
array(['1', 'a', '2.5'], dtype='<U32')
```

COMMON TRAP

One string turns everything into text. `np.array([1, 'a', 2.5])` gives `['1' 'a' '2.5']`: the numbers are now strings and you can no longer add or multiply them. The `<U32` in the output means "Unicode text, up to 32 characters". NumPy gives no warning, so a stray string in a CSV column can silently ruin a numeric array.

MEMORY TRICK

Upcasting only goes **up** the ladder: bool → int → float → string. Think of a lift that carries every value to the highest floor present.

2.3 Choosing the type yourself: the dtype keyword

Add `dtype=` when you want to decide the type instead of letting NumPy guess. **MUST KNOW**

PYTHON • type this in Colab

```
b = np.array([1, 2, 3, 4], dtype='float32')
b
```

▶ OUTPUT

```
array([1., 2., 3., 4.], dtype=float32)
```

PYTHON • type this in Colab

```
print(b.dtype)
np.array([1, 2, 3], dtype=float)
```

▶ OUTPUT

```
float32
array([1., 2., 3.])
```

You can write the type as a **string** (`'float32'`) or as a Python/NumPy type (`float`, `np.float32`). All of these work and mean the same thing.

WHY IT MATTERS

Deep-learning libraries such as PyTorch use **float32** by default, while NumPy defaults to **float64**. float32 uses half the memory. You will often write `dtype='float32'` when preparing data for a neural network.

2.4 Multi-dimensional arrays: a list of lists

A **list of lists** becomes a 2D array. Each inner list is one **row**.

PYTHON • type this in Colab

```
np.array([range(i, i + 3) for i in [2, 4, 6]])
```

▶ OUTPUT

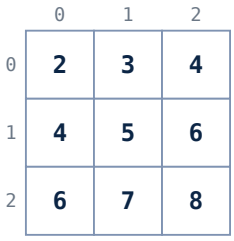
```
array([[2, 3, 4],
       [4, 5, 6],
       [6, 7, 8]])
```

PYTHON • type this in Colab

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])
M
```

▶ OUTPUT

```
array([[1, 2, 3],
       [4, 5, 6]])
```



3 rows x 3 columns



M: 2 rows x 3 columns

WHY IT MATTERS

A dataset is a 2D array: **each row is one sample** (one student, one image, one house) and **each column is one feature** (marks, price, age). Getting comfortable with rows and columns now pays off in every ML algorithm.

2.5 Creating arrays from scratch

For big arrays you never type the values by hand. You ask NumPy to build them. Here is the full toolbox from the book, with how much you will actually use each one:

Function	Builds	Typical use in AI/ML	Priority
`np.zeros`	All 0	Placeholders, initialising	MUST KNOW
`np.ones`	All 1	Bias columns, masks	MUST KNOW
`np.full`	All one chosen value	Constant fills	GOOD TO KNOW
`np.arange`	Sequence by step size	Indexes, loops, x-axis values	MUST KNOW
`np.linspace`	N evenly spaced points	Plots, ranges, thresholds	MUST KNOW
`np.random.random`	Uniform random in [0, 1)	Fake data, shuffling	GOOD TO KNOW
`np.random.normal`	Bell-curve random	Weight initialisation, noise	MUST KNOW
`np.random.randint`	Random whole numbers	Fake labels, dice, sampling	GOOD TO KNOW
`np.eye`	Identity matrix	Linear algebra	MUST KNOW
`np.empty`	Uninitialised (junk) values	Rare speed trick	LOOK UP

Filled arrays: zeros, ones, full

PYTHON • type this in Colab

```
np.zeros(10, dtype=int)
```

▶ OUTPUT

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

PYTHON • type this in Colab

```
np.ones((3, 5), dtype=float)
```

▶ OUTPUT

```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

PYTHON • type this in Colab

```
np.full((3, 5), 3.14)
```

▶ OUTPUT

```
array([[3.14, 3.14, 3.14, 3.14, 3.14],
       [3.14, 3.14, 3.14, 3.14, 3.14],
       [3.14, 3.14, 3.14, 3.14, 3.14]])
```

	0	1	2	3
0	0	0	0	0
1	0	0	0	0

np.zeros((2, 4), dtype=int)

	0	1	2	3
0	1.	1.	1.	1.
1	1.	1.	1.	1.

np.ones((2, 4))

	0	1	2	3
0	3.14	3.14	3.14	3.14
1	3.14	3.14	3.14	3.14

np.full((2, 4), 3.14)

COMMON TRAP

The shape must be ONE argument: a tuple. `np.zeros(3, 4)` is wrong. The second argument is the dtype, so NumPy tries to read 4 as a type name:

`TypeError: Cannot interpret '4' as a data type`

The correct call is `np.zeros((3, 4))` with **double brackets**. The same rule holds for ones, full, empty and the random functions.

MEMORY TRICK

A 2D shape is always **(rows, columns)**. Say it as "rows by columns": (3, 5) is 3 rows and 5 columns.

Sequences: arange and linspace

Both make a line of numbers, but they ask you for different information. This is the most confused pair in the whole chapter.

PYTHON • type this in Colab

```
np.arange(0, 20, 2)
```

▶ OUTPUT

```
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

`np.arange(start, stop, step)` works like Python's `range`. It starts at **start**, adds **step** again and again and stops **before** it reaches **stop**. The stop value is **excluded**.

PYTHON • type this in Colab

```
np.arange(5)
```

▶ OUTPUT

```
array([0, 1, 2, 3, 4])
```

PYTHON • type this in Colab

```
np.arange(2, 8)
```

▶ OUTPUT

```
array([2, 3, 4, 5, 6, 7])
```

PYTHON • type this in Colab

```
np.arange(0, 1, 0.25)
```

▶ OUTPUT

```
array([0.  , 0.25, 0.5 , 0.75])
```

PYTHON • type this in Colab

```
np.linspace(0, 1, 5)
```

▶ OUTPUT

```
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
```

`np.linspace(start, stop, num)` gives you exactly **num** points, evenly spaced, and the stop value is **included**. NumPy works out the step for you.

PYTHON • type this in Colab

```
np.linspace(1, 10, 4)
```

▶ OUTPUT

```
array([ 1.,  4.,  7., 10.])
```



np.arange(1, 10, 3): stop 10 excluded *np.linspace(1, 10, 4): stop 10 included*

	np.arange(1, 10, 3)	np.linspace(1, 10, 4)
3rd argument means	the **step** (gap between numbers)	the **count** (how many numbers)
Stop value	**Excluded**	**Included**
Result	`[1 4 7]`	`[ 1. 4. 7. 10.]`
Element type	int (if you give ints)	always float

MEMORY TRICK

arange = A Rule of step (you give the step; stop excluded).
linspace = Lots of points (you give the count; stop included). If you need to hit an exact end value, use linspace.

NOTE

With decimal steps such as `np.arange(0, 1, 0.1)` tiny rounding errors can make the length surprising. When your step is a decimal, prefer `np.linspace` because you control the exact number of points.

Random arrays and the seed

NumPy can fill arrays with random numbers. The random numbers are not truly random: they come from a formula that starts from a number called the **seed**. Same seed gives the same "random" numbers every time, which is exactly what you want when an experiment must be **reproducible**. **MUST KNOW**

PYTHON • type this in Colab

```
np.random.seed(42)
np.random.random((2, 3))
```

▶ OUTPUT

```
array([[0.37454012, 0.95071431, 0.73199394],
       [0.59865848, 0.15601864, 0.15599452]])
```

`np.random.random(shape)` gives uniformly spread numbers between 0 (included) and 1 (excluded).

PYTHON • type this in Colab

```
np.random.seed(42)
np.random.normal(0, 1, (3, 3))
```

▶ OUTPUT

```
array([[ 0.49671415, -0.1382643 ,  0.64768854],
       [ 1.52302986, -0.23415337, -0.23413696],
       [ 1.57921282,  0.76743473, -0.46947439]])
```

`np.random.normal(mean, std, shape)` gives numbers that follow the **bell curve**: most values are near the mean, few are far away. With mean 0 and std 1 you get the "standard normal" used everywhere in ML. You can change the centre and spread:

PYTHON • type this in Colab

```
np.random.seed(0)
np.random.normal(50, 10, 5)
```

▶ OUTPUT

```
array([67.64052346, 54.00157208, 59.78737984, 72.40893199, 68.6755799 ])
```

PYTHON • type this in Colab

```
np.random.seed(1)
np.random.randint(0, 10, (3, 3))
```

▶ OUTPUT

```
array([[5, 8, 9],
       [5, 0, 0],
       [1, 7, 6]])
```

`np.random.randint(low, high, shape)` gives whole numbers from **low up to high - 1**. The high value is **excluded**, just like arange.

PYTHON • type this in Colab

```
np.random.seed(2)
dice = np.random.randint(1, 7, size=10)
dice
```

▶ OUTPUT

```
array([1, 6, 1, 4, 3, 4, 1, 3, 2, 4])
```

COMMON TRAP

Dice need high = 7, not 6. `np.random.randint(1, 6, 10)` can never produce a 6 because the high value is excluded.

INTERVIEW FLAG

"Why do we set a random seed?"

So that results are reproducible. Anyone who runs the same code gets the same random numbers, so a model's accuracy can be compared fairly between runs and between people.

VERSION NOTE

The book uses `np.random.seed` and `np.random.random`. They still work and you will see them everywhere. Newer code often uses a **generator object** instead. Recognise both styles:

PYTHON • type this in Colab

```
rng = np.random.default_rng(0)
print(rng.random((2, 3)))
print(rng.normal(0, 1, 3))
print(rng.integers(0, 10, 5))
```

▶ OUTPUT

```
[[0.63696169 0.26978671 0.04097352]
 [0.01652764 0.81327024 0.91275558]]
[ 1.30400005  0.94708096 -0.70373524]
[5 9 2 8 6]
```

Notice the matching names: `random` → `rng.random`, `normal` → `rng.normal`, `randint` → `rng.integers`. Either style is fine for now.

Identity matrix: eye

PYTHON • type this in Colab

```
np.eye(3)
```

▶ OUTPUT

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

PYTHON • type this in Colab

```
np.eye(3, dtype=int)
```

▶ OUTPUT

```
array([[1, 0, 0],
       [0, 1, 0],
       [0, 0, 1]])
```

	0	1	2	3
0	**1**	**0**	**0**	**0**
1	**0**	**1**	**0**	**0**
2	**0**	**0**	**1**	**0**
3	**0**	**0**	**0**	**1**

np.eye(4, dtype=int): 1s on the diagonal

The identity matrix is the matrix version of the number 1: multiplying any matrix by it changes nothing. You will meet it repeatedly in your Linear Algebra videos (identity transformation, inverse). Note that `np.eye(3)` takes just one number because the matrix is square.

Uninitialised arrays: empty

PYTHON • type this in Colab

```
np.empty(3)
```

▶ OUTPUT

```
array([...])   <- three unpredictable numbers, different on different runs
```

`np.empty` reserves memory but does **not** write anything into it, so you see whatever junk was already there. It is slightly faster than `np.zeros` but dangerous.

IMPORTANT Avoid `np.empty` unless you will overwrite every single slot straight away. Reading a slot you never wrote gives garbage that looks like real data. When in doubt, use `np.zeros`.

✎ PRACTICE PROBLEMS | Part 2: Creating Arrays

Q1 PREDICT THE OUTPUT

What is the array and what is its dtype? Explain using the upcasting ladder.

PYTHON • type this in Colab

```
a = np.array([1, 2.5, True])
print(a)
print(a.dtype)
```

Q2 PREDICT THE OUTPUT

What does this print, and why can you no longer calculate with the result?

PYTHON • type this in Colab

```
b = np.array([1, 'a', 2.5])
print(b)
print(b.dtype)
```

Q3 BUILD IT

Write one line that builds a **2x3** array from nested lists with dtype **float32**. Print it and its dtype.

Q4 PREDICT THE OUTPUT

Predict both lines.

PYTHON • type this in Colab

```
print(np.arange(1, 10, 3))
print(np.linspace(1, 10, 4))
```

Q5 CALCULATE

(a) How many numbers does **np.arange(0, 10, 2)** make, and what is the last one? (b) How many does **np.linspace(0, 10, 6)** make, and what is the last one? Verify in Colab.

PYTHON • type this in Colab

```
a = np.arange(0, 10, 2)
b = np.linspace(0, 10, 6)
print(a, len(a))
print(b, len(b))
```

Q6 BUILD IT

Create a **4x4** array of normally distributed random values (mean 0, std 1). Use seed 0 so your answer is repeatable.

Q7 FIND THE BUGS

Find the mistake in each line. (a) is a crash, (b) is a crash, (c) runs fine but does not do what the comment says.

PYTHON • type this in Colab

```
a = np.zeros(3, 4)                    # want a 3x4 array of zeros
b = np.linspace(0, 1, 0.25)           # want points 0.25 apart
c = np.random.randint(1, 6, 10)       # want 10 dice throws (1 to 6)
```

Q8 BUILD IT

Write one line for each: (a) a 3x3 identity matrix of whole numbers; (b) 10 evenly spaced numbers from -1 to 1; (c) a 2x5 array filled with 7s.

Q9 THINK IT THROUGH

Why is **np.empty** risky, and what would you use instead if you are unsure?

Q10 **THEORY**

[4 marks] State two differences between **np.arange** and **np.linspace**, and explain why we set a random seed.

PART 3

3 NumPy Data Types

Block 1, last section: the standard NumPy types, their limits and their dangers

IN THIS PART YOU WILL LEARN

- ✓ How to read a type name like **float32** or **uint8**
- ✓ The full table of NumPy types and which ones matter in AI/ML
- ✓ Two ways to set a dtype, and how to check the limits of any type
- ✓ Silent **overflow**: the bug that corrupts images without any error
- ✓ How the dtype controls memory, and how **astype** changes it

3.1 Why types matter so much in NumPy

Because a NumPy array holds one fixed type, you must know the types and their **limits**. NumPy is built in C, so its types look like the types of C and Fortran: they have a fixed size in memory and therefore a fixed range of values they can hold.

How to read a type name

A type name is a **family** followed by the **number of bits**. Eight bits make one byte, so you can read the memory use straight off the name:

Name	Family	Bits	Bytes per item
`int8`	whole numbers (positive and negative)	8	1
`int32`	whole numbers (positive and negative)	32	4
`uint8`	**u**nsigned whole numbers (0 and up only)	8	1
`float32`	decimal numbers	32	4
`float64`	decimal numbers	64	8

MEMORY TRICK

u = unsigned = no minus sign. Because it never stores negatives, uint8 uses its 8 bits for 0 to 255, while int8 splits the same 8 bits into -128 to 127.

3.2 The standard NumPy types (from the book)

Type	Description	Range / precision
`bool`	True or False, stored as one byte	True / False
`int8`	Byte	-128 to 127
`int16`	Integer	-32,768 to 32,767
`int32`	Integer	-2,147,483,648 to 2,147,483,647
`int64`	Integer	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`uint8`	Unsigned integer	0 to 255
`uint16`	Unsigned integer	0 to 65,535
`uint32`	Unsigned integer	0 to 4,294,967,295
`uint64`	Unsigned integer	0 to 18,446,744,073,709,551,615
`float16`	Half precision float	1 sign bit, 5 exponent bits, 10 mantissa bits

Type	Description	Range / precision
`float32`	Single precision float	1 sign bit, 8 exponent bits, 23 mantissa bits
`float64`	Double precision float	1 sign bit, 11 exponent bits, 52 mantissa bits
`complex64`	Complex number	two 32-bit floats
`complex128`	Complex number	two 64-bit floats

VERSION NOTE

The book's table also lists the shorthand names `bool_`, `int_`, `intc`, `intp`, `float_` and `complex_`. In NumPy 2.0 the names `float_` and `complex_` were **removed**. Write `float64` and `complex128` instead. The plain Python names `int`, `float` and `bool` also work as dtypes.

Which types will you actually meet?

Type	Where you meet it in AI/ML	Priority
`float32`	**Deep-learning default** (PyTorch, TensorFlow). Half the memory of float64.	MUST KNOW
`float64`	**NumPy's default** for decimals. Used in classic ML and statistics.	MUST KNOW
`int64`	NumPy's default for whole numbers. Class labels, indices, counts.	MUST KNOW
`uint8`	**Image pixels** (0 to 255). The classic overflow victim.	MUST KNOW
`bool`	Masks and filters (comes in a later topic).	GOOD TO KNOW
`float16`	Memory-saving training on big models.	GOOD TO KNOW
`int8 / int16 / int32`	Compressed or quantised data.	LOOK UP
`complex64 / complex128`	Signal processing. Rare in ML.	LOOK UP

INTERVIEW FLAG

"Why does deep learning use float32 instead of float64?" It halves memory use and speeds up computation, and neural networks do not need the extra precision of float64.

3.3 Setting the dtype: two ways

You can name the type as a **string** or as a **NumPy object**. Both give the same result.

PYTHON • type this in Colab

```
np.zeros(10, dtype='int16')
```

▶ OUTPUT

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int16)
```

PYTHON • type this in Colab

```
np.zeros(10, dtype=np.int16)
```

▶ OUTPUT

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int16)
```

What type do you get if you do not choose?

PYTHON • type this in Colab

```
print(np.array([1, 2, 3]).dtype)
print(np.array([1.0, 2.0]).dtype)
print(np.array([True, False]).dtype)
```

▶ OUTPUT

```
int64
float64
bool
```

COMMON TRAP The default whole-number type is **int64** on most modern systems, but older NumPy versions on Windows used **int32**. Never assume: print `.dtype` whenever the size matters.

3.4 Checking the limits of a type

You do not need to memorise the ranges. NumPy can tell you with `np.iinfo` (integers) and `np.finfo` (floats). **LOOK UP**

PYTHON • type this in Colab

```
info = np.iinfo(np.int8)
print(info.min, info.max)
info = np.iinfo(np.uint8)
print(info.min, info.max)
print(np.finfo(np.float16).max)
```

▶ OUTPUT

```
-128 127
0 255
6.55e+04
```

3.5 Overflow: the silent bug

Every type has a maximum. When a calculation goes past it, NumPy does **not** raise an error. The value **wraps around** to the other end, like an odometer rolling from 999 back to 000.

PYTHON • type this in Colab

```
a = np.array([200], dtype=np.uint8)
print(a + 100)
```

▶ OUTPUT

```
[44]
```

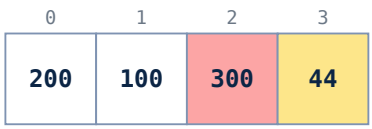
uint8 can hold only 0 to 255. The true answer 300 does not fit, so it wraps: 300 - 256 = **44**. Another example with two arrays:

PYTHON • type this in Colab

```
p = np.array([250, 10], dtype=np.uint8)
q = np.array([10, 10], dtype=np.uint8)
print(p + q)
```

▶ OUTPUT

```
[ 4 20]
```



200 + 100 should be 300, but a uint8 box only reaches 255

ANALOGY A **car odometer** with three digits shows 999. Drive one more kilometre and it shows 000, not 1000. A uint8 "odometer" has 256 positions (0 to 255), so after 255 it rolls back to 0.

WHY IT MATTERS Images are stored as uint8. Adding brightness to an image with plain + makes bright pixels wrap to dark ones and produces ugly black speckles, with no error message. The fix is to convert to a bigger type first (next topic: astype).

VERSION NOTE In NumPy 2, adding a Python number that *itself* does not fit the array type is refused loudly:

PYTHON • type this in Colab

```
a = np.array([200], dtype=np.uint8)
a + 300
```

▶ OUTPUT

```
OverflowError: Python integer 300 out of bounds for uint8
```

But a result that grows past the limit (like 200 + 100 above) still wraps silently. Do not rely on the error to protect you.

3.6 The dtype controls memory

PYTHON • type this in Colab

```
n = 1_000_000
print("float64:", np.zeros(n, dtype='float64').nbytes, "bytes")
print("float32:", np.zeros(n, dtype='float32').nbytes, "bytes")
print("float16:", np.zeros(n, dtype='float16').nbytes, "bytes")
print("uint8  :", np.zeros(n, dtype='uint8').nbytes, "bytes")
```

▶ OUTPUT

```
float64: 8000000 bytes
float32: 4000000 bytes
float16: 2000000 bytes
uint8  : 1000000 bytes
```

Same one million values, eight times difference between the biggest and smallest. This is why picking the right dtype matters when data gets large.

3.7 Changing the type of an existing array: astype

To convert an array you already have, call `.astype(new_type)`. It returns a **new** array and leaves the original alone.

MUST KNOW

PYTHON • type this in Colab

```
f = np.array([1.7, 2.9, -1.7])
print(f.astype(int))
print(f.astype('float32'))
```

▶ OUTPUT

```
[ 1  2 -1]
[ 1.7  2.9 -1.7]
```

COMMON TRAP **Decimal to integer chops, it does not round.** 1.7 becomes 1 and -1.7 becomes -1 (the decimal part is simply thrown away, towards zero). If you want rounding, call `np.round(f)` first.

PYTHON • type this in Colab

```
pixels = np.array([100, 200, 250], dtype=np.uint8)
safe = pixels.astype(np.int16) + 50
print(safe)
```

▶ OUTPUT

```
[150 250 300]
```

Converting to the larger `int16` *before* adding gives the correct 300 instead of the wrapped 44.

NOTE `astype` is a bonus: the book does not cover it in these two blocks, but you will use it all the time to move between float64, float32 and int.

✎ PRACTICE PROBLEMS | Part 3: Data Types

Q1 READ & DECODE

Explain in plain words: (a) **float32** (b) **uint8** (c) **int16** (d) **complex128**.

Q2 CALCULATE

(a) How many different values can an **int8** hold? (b) What is the largest **uint16**? Check with **np.iinfo**.

Q3 PREDICT THE OUTPUT

What does this print and why?

PYTHON • type this in Colab

```
x = np.array([250], dtype=np.uint8)
print(x + 10)
```

Q4 CALCULATE

A colour image is 224 x 224 pixels with 3 colour channels. How many bytes does it need as **uint8**, and how many as **float32**? Check with **np.zeros**.

Q5 THINK IT THROUGH

Why do deep-learning libraries default to **float32** rather than **float64**? Give two reasons.

Q6 PREDICT THE OUTPUT

What does **astype(int)** do to these numbers? Does it round?

PYTHON • type this in Colab

```
print(np.array([1.7, 2.9, -1.7]).astype(int))
```

Q7 FIND THE BUGS

A student brightens a tiny 3-pixel image by 50 and expects **[150 250 300]**. Explain what went wrong and fix it.

PYTHON • type this in Colab

```
pixels = np.array([100, 200, 250], dtype=np.uint8)
brighter = pixels + 50
print(brighter)
```

Q8 THEORY

[4 marks] What is overflow in NumPy? Explain with an example why it is dangerous and how to avoid it.

PART 4

4 Array Attributes

Block 2, first section: what an array can tell you about itself

IN THIS PART YOU WILL LEARN

- ✓ The difference between an attribute and a method (no brackets!)
- ✓ shape, ndim, size, dtype, itemsize and nbytes, one by one
- ✓ How to read a shape tuple like (3, 4, 5)
- ✓ The golden habit: print **.shape** before every operation

4.1 What is an attribute?

An **attribute** is a fact that an array stores about itself. You read it by writing a dot and the name, **without brackets**: `x.shape`. A **method** (like `x.copy()`) is an action and needs brackets. Mixing these up is the most common beginner error here.

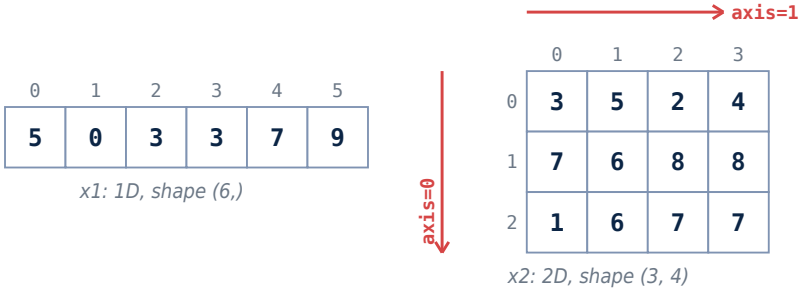
The book defines three random arrays: one-dimensional, two-dimensional and three-dimensional. We seed the generator so the numbers are the same every time.

PYTHON • type this in Colab

```
np.random.seed(0)                          # seed for reproducibility
x1 = np.random.randint(10, size=6)         # one-dimensional
x2 = np.random.randint(10, size=(3, 4))    # two-dimensional
x3 = np.random.randint(10, size=(3, 4, 5)) # three-dimensional
print(x1)
print(x2)
```

▶ OUTPUT

```
[5 0 3 3 7 9]
[[3 5 2 4]
 [7 6 8 8]
 [1 6 7 7]]
```



The 3D array `x3` is easiest to picture as **3 layers**, each layer a table of 4 rows and 5 columns:

8	1	5	9	8
9	4	3	0	3
5	0	2	3	8
1	3	3	3	7

x3[0]: 4 rows x 5 columns

0	1	9	9	0
4	7	3	2	7
2	0	0	4	5
5	6	8	4	1

x3[1]: 4 rows x 5 columns

4	9	8	1	1
7	9	9	3	6
7	2	0	3	5
9	4	4	6	4

x3[2]: 4 rows x 5 columns

4.2 shape, ndim, size and dtype

PYTHON • type this in Colab

```
print("x3 ndim :", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size :", x3.size)
print("dtype   :", x3.dtype)
```

▶ OUTPUT

```
x3 ndim : 3
x3 shape: (3, 4, 5)
x3 size : 60
dtype   : int64
```

Attribute	Meaning	Value for x3	Priority
`shape`	Size of each dimension, as a tuple	`(3, 4, 5)`	**MUST KNOW**
`ndim`	Number of dimensions (how many numbers in the shape)	`3`	**GOOD TO KNOW**
`size`	Total number of elements (multiply the shape)	`60`	**GOOD TO KNOW**
`dtype`	Data type of every element	`int64`	**MUST KNOW**
`itemsize`	Bytes used by ONE element	`8`	**LOOK UP**
`nbytes`	Bytes used by the WHOLE array	`480`	**GOOD TO KNOW**

MEMORY TRICK

size = the product of the shape. For (3, 4, 5): 3 x 4 x 5 = 60. And **ndim = the length of the shape**: three numbers, so 3 dimensions.

4.3 itemsize and nbytes

PYTHON • type this in Colab

```
print("itemsize:", x3.itemsize, "bytes")
print("nbytes  :", x3.nbytes, "bytes")
print(x3.itemsize * x3.size == x3.nbytes)
```

▶ OUTPUT

```
itemsize: 8 bytes
nbytes  : 480 bytes
True
```

In general **nbytes = itemsize x size**: 8 bytes x 60 elements = 480 bytes.

COMMON TRAP

Never assume the dtype when calculating bytes. A question that says "assume int32" is different from what Colab shows by default. Without a dtype, `np.random.randint` gives **int64** (8 bytes each), so a (5, 3, 2) array has 30 x 8 = 240 bytes. Set `dtype=np.int32` and the same shape needs only 30 x 4 = 120 bytes.

4.4 How to read a shape tuple

Shape	ndim	In words	AI/ML example
`(6,)`	1	six numbers in a line	A single feature vector
`(3, 4)`	2	3 rows, 4 columns	A dataset: 3 samples, 4 features
`(3, 4, 5)`	3	3 layers of (4 x 5)	A stack of tables
`(100, 28, 28)`	3	100 pictures, each 28 x 28 pixels	100 handwritten-digit images
`(32, 224, 224, 3)`	4	32 pictures, each 224 x 224 with 3 colours	One batch of colour photos

NOTE A 1D shape is written **(6,)** with a trailing comma. In Python a tuple with one item needs the comma, otherwise `(6)` is just the number 6 in brackets.

WHY IT MATTERS **The #1 bug in ML code is a shape mismatch.** Look at what happens when you try to multiply a 3x4 matrix by a 5x2 matrix (matrix multiplication, written `@`, is coming in your Linear Algebra videos):

PYTHON • type this in Colab

```
A = np.ones((3, 4))
B = np.ones((5, 2))
print(A.shape, B.shape)
A @ B
```

▶ OUTPUT

```
(3, 4) (5, 2)
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc
    signature (n?,k),(k,m?)->(n?,m?) (size 5 is different from 4)
```

The inner numbers (4 and 5) must match. **The habit that prevents most of these:** print `.shape` of every array before you operate on it.

INTERVIEW FLAG **"What is the difference between shape, size and ndim?"** shape is the tuple of dimension lengths, size is the total number of elements (the product of the shape), ndim is how many dimensions there are (the length of the shape).

✎ PRACTICE PROBLEMS | Part 4: Array Attributes

Q1 READ & DECODE

An array has shape **(5, 3, 2)**. Without running anything, state its ndim and size, and describe it in words.

Q2 CALCULATE

The array of Q1 has dtype **int32**. Find its itemsize and nbytes by hand, then verify by building it with **dtype=np.int32**.

Q3 PREDICT THE OUTPUT

Predict the five numbers printed.

PYTHON • type this in Colab

```
x = np.zeros((4, 6), dtype='float32')
print(x.ndim, x.shape, x.size, x.itemsize, x.nbytes)
```

Q4 CALCULATE

A dataset holds **100 grayscale images** of 28 x 28 pixels stored as **uint8**. Give the shape, the size and the nbytes.

Q5 **FIND THE BUGS**

The code below crashes. Why? Fix it.

PYTHON • type this in Colab

```
x = np.arange(12)
print(x.shape())
```

Q6 **THINK IT THROUGH**

Why does the shape of a 1D array print as **(6,)** and not **(6)**?

Q7 **CALCULATE**

For a **10 x 10** float64 array, find size, itemsize and nbytes, then confirm that nbytes = itemsize x size.

Q8 **THEORY**

[4 marks] Define shape, ndim, size and dtype with one example each. Why is printing the shape a good habit?

PART 5

5 Indexing

Block 2: getting and setting a single element

IN THIS PART YOU WILL LEARN

- ✓ Positive and negative indexes in 1D arrays
- ✓ The [row, column] form for 2D arrays
- ✓ How to change a value, and the silent truncation trap
- ✓ What an IndexError means

5.1 Indexing a 1D array

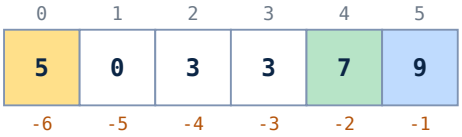
If you know Python lists, you already know this. Counting starts at **0**. Negative numbers count backwards from the end: **-1 is the last item**. **MUST KNOW**

PYTHON • type this in Colab

```
x1 = np.array([5, 0, 3, 3, 7, 9])
print(x1[0])
print(x1[4])
print(x1[-1])
print(x1[-2])
```

▶ OUTPUT

```
5
7
9
7
```



top row = positive index, bottom row = negative index

MEMORY TRICK

Positive indexes start at **0** on the left. Negative indexes start at **-1** on the right. So **x[-1]** is always the last item, whatever the length.

PYTHON • type this in Colab

```
x1[6]
```

▶ OUTPUT

```
IndexError: index 6 is out of bounds for axis 0 with size 6
```

Going past the end raises an **IndexError**: the array has 6 items, so the valid indexes are 0 to 5 (or -6 to -1).

5.2 Indexing a 2D array: [row, column]

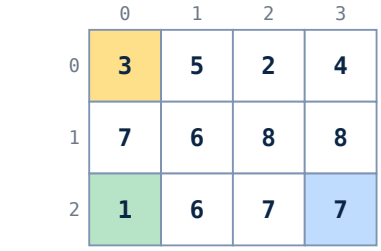
In a multi-dimensional array you give **one index per dimension**, separated by commas. For 2D it is always **[row, column]**. **MUST KNOW**

PYTHON • type this in Colab

```
x2 = np.array([[3, 5, 2, 4],
               [7, 6, 8, 8],
               [1, 6, 7, 7]])
print(x2[0, 0])
print(x2[2, 0])
print(x2[2, -1])
```

▶ OUTPUT

```
3
1
7
```



x2[0,0] yellow x2[2,0] green x2[2,-1] blue

NOTE

`x2[0][0]` also works, but it is two steps (first pick row 0, then pick item 0 of that row). The comma form `x2[0, 0]` is the NumPy standard, and it is the form slicing uses in the next topic.

5.3 Changing values, and the truncation trap

PYTHON • type this in Colab

```
x2[0, 0] = 12
print(x2)
```

▶ OUTPUT

```
[[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

Because an array has a **fixed type**, it cannot hold a value of a different kind. Put a decimal into an integer array and NumPy chops it, with **no error**:

PYTHON • type this in Colab

```
x1[0] = 3.14159
print(x1)
```

▶ OUTPUT

```
[3 0 3 3 7 9]
```

COMMON TRAP

Silent truncation. The 3.14159 became 3 and nothing warned you. It chops towards zero (7.9 becomes 7, not 8). In ML this can wipe out a dataset: store probabilities such as 0.87 in an integer array and every one turns into 0. If you need decimals, create the array as float (`dtype=float`).

✎ PRACTICE PROBLEMS | Part 5: Indexing

Q1 **PREDICT THE OUTPUT**

Predict the three values.

PYTHON • type this in Colab

```
x = np.array([[10, 20, 30, 40],
              [50, 60, 70, 80],
              [90, 100, 110, 120]])
print(x[1, 2], x[-1, -1], x[0, -2])
```

Q2 **PREDICT THE OUTPUT**

What is **x[2, 0]** after the assignment, and why?

PYTHON • type this in Colab

```
x = np.array([[10, 20, 30, 40],
              [50, 60, 70, 80],
              [90, 100, 110, 120]])
x[2, 0] = 7.9
print(x[2, 0])
```

Q3 **FIND THE BUGS**

The code crashes. Explain the error message and give the valid index range.

PYTHON • type this in Colab

```
x = np.array([4, 8, 15, 16, 23, 42])
print(x[6])
```

Q4 **BUILD IT**

From the array **x = np.arange(1, 13).reshape(3, 4)** print (a) the value 12 using a **negative** index, (b) the value 5 using **positive** indexes.

Q5 **THINK IT THROUGH**

You store the probabilities **[0.87, 0.12, 0.99]** in an array created with **np.zeros(3, dtype=int)** by assigning one by one. What do you get? How do you avoid the problem?

6 Slicing

PART 6

Block 2: getting a whole range with start:stop:step

IN THIS PART YOU WILL LEARN

- ✓ The pattern x[start:stop:step] and its defaults
- ✓ Reversing with a negative step
- ✓ Slicing 2D arrays one dimension at a time
- ✓ Getting a single row or a single column
- ✓ Why slicing never raises an IndexError

6.1 The slicing pattern

Indexing gets **one** item. Slicing gets a **range**, using a colon. The pattern is:

```
x[ start : stop : step ]
```

Part	Meaning	If you leave it out
start	first index to take (**included**)	0 (the beginning)
stop	where to stop (**excluded**)	the end of the array
step	jump between items	1

MEMORY TRICK

Slicing follows the same rule as `np.arange` and `range`: **start included, stop excluded**. `x[4:7]` gives indexes 4, 5, 6.

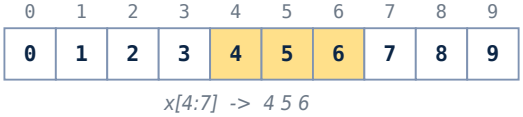
One-dimensional slices

PYTHON • type this in Colab

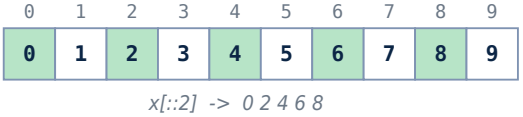
```
x = np.arange(10)
print(x[:5])      # first five
print(x[5:])      # from index 5 to the end
print(x[4:7])     # a middle piece
print(x[::2])     # every other element
print(x[1::2])    # every other, starting at index 1
```

▶ OUTPUT

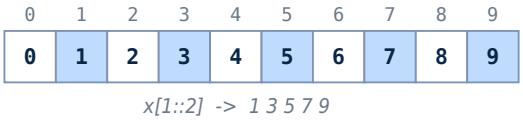
```
[0 1 2 3 4]
[5 6 7 8 9]
[4 5 6]
[0 2 4 6 8]
[1 3 5 7 9]
```



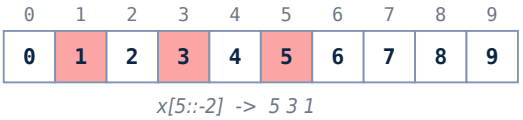
x[4:7] -> 4 5 6



x[::2] -> 0 2 4 6 8



x[1::2] -> 1 3 5 7 9



x[5::-2] -> 5 3 1

Negative step: reversing

PYTHON • type this in Colab

```
print(x[::-1])     # everything, reversed
print(x[5::-2])    # from index 5, backwards, every other
```

▶ OUTPUT

```
[9 8 7 6 5 4 3 2 1 0]
[5 3 1]
```

COMMON TRAP **With a negative step the defaults swap.** Start defaults to the *end* and stop to the *beginning*, which is why `x[::-1]` reverses the whole array. This is the one confusing case: if in doubt, run it and trace it on paper.

COMMON TRAP **::2 and 1::2 are different.** `x[::2]` starts at index 0 and takes 0, 2, 4... `x[1::2]` starts at index 1 and takes 1, 3, 5... A missing start always means 0.

6.2 Slicing 2D arrays

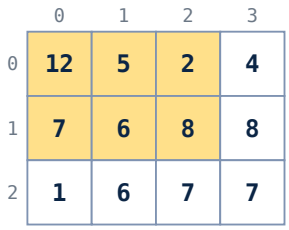
Give **one slice per dimension**, separated by a comma: **[rows-slice, columns-slice]**. **MUST KNOW**

PYTHON • type this in Colab

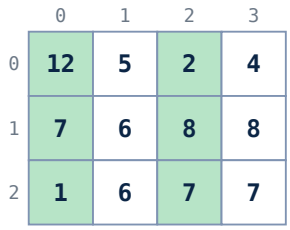
```
x2 = np.array([[12, 5, 2, 4],
               [ 7, 6, 8, 8],
               [ 1, 6, 7, 7]])
print(x2[:2, :3])      # two rows, three columns
print(x2[:3, ::2])     # all rows, every other column
print(x2[::-1, ::-1])  # rows and columns both reversed
```

▶ OUTPUT

```
[[12  5  2]
 [ 7  6  8]]
[[12  2]
 [ 7  8]
 [ 1  7]]
[[ 7  7  6  1]
 [ 8  8  6  7]
 [ 4  2  5 12]]
```



x2[:2, :3]



x2[:3, ::2]

6.3 Getting one row or one column

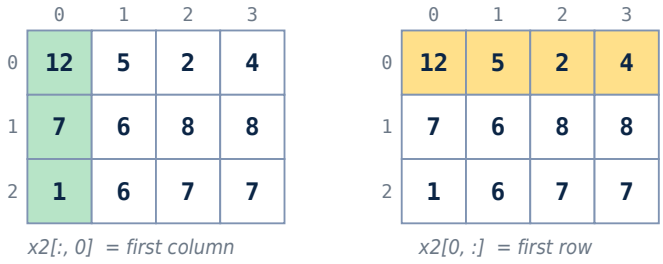
A lone colon `:` means "everything along this dimension". Combine it with a single index to pick a row or a column. **MUST KNOW**

PYTHON • type this in Colab

```
print(x2[:, 0])    # first COLUMN
print(x2[0, :])    # first ROW
print(x2[0])       # shorter way to write the first row
```

▶ OUTPUT

```
[12  7  1]
[12  5  2  4]
[12  5  2  4]
```



WHY IT MATTERS In an ML dataset `X[:, 0]` means "the first **feature** of every sample" and `X[0]` means "the first **sample** with all its features". You will type these two patterns constantly.

6.4 Slicing never crashes on range

PYTHON • type this in Colab

```
print(x[5:100])
print(x[100:])
```

▶ OUTPUT

```
[5 6 7 8 9]
[]
```

Unlike indexing, a slice that goes past the end is simply cut short, and an empty slice gives an empty array. No IndexError.

✎ PRACTICE PROBLEMS | Part 6: Slicing

Q1 PREDICT THE OUTPUT

Predict all four lines, then run them.

PYTHON • type this in Colab

```
x = np.arange(10)
print(x[2:6])
print(x[::3])
print(x[-3:])
print(x[::-2])
```

Q2 BUILD IT

For the 4x5 array below write the slice for: (a) the last column as a 1D array; (b) the top-right 2x2 block; (c) columns 0, 2 and 4 of every row.

PYTHON • type this in Colab

```
x = np.arange(1, 21).reshape(4, 5)
print(x)
```

Q3 FIND THE BUGS

A student wants **columns 0, 2, 4** and writes the line below. The code runs, but the answer is wrong. Why?

PYTHON • type this in Colab

```
x = np.arange(1, 21).reshape(4, 5)
print(x[:, 1::2])
```

Q4 READ & DECODE

Explain in words: (a) **x[::-1]** (b) **x[1::2]** (c) **x[:, 0]** (d) **x[-3:]**.

Q5 PREDICT THE OUTPUT

What are the shapes of these three results?

PYTHON • type this in Colab

```
x = np.arange(12).reshape(3, 4)
print(x[0].shape)
print(x[:, 0].shape)
print(x[:2, :2].shape)
```

Q6 THINK IT THROUGH

What happens with **np.arange(10)[5:100]** and why does it not raise an IndexError?

Q7 BUILD IT

Using one slice each, get from **x = np.arange(10)**: (a) the numbers 7, 8, 9; (b) the numbers 9, 7, 5, 3, 1; (c) all items except the first and last.

Q8 THEORY

[3 marks] Write the pattern of slicing, give the default for each part, and explain what happens when the step is negative.

PART 7

7 Views vs Copies

Block 2: the most important idea in the chapter

IN THIS PART YOU WILL LEARN

- ✓ Why a slice is a window onto the same data, not a new array
- ✓ How changing a view silently changes the original
- ✓ How to make a real independent copy with .copy()
- ✓ How to test it with np.shares_memory
- ✓ Why this bug shows up in real ML code

7.1 Slices are views

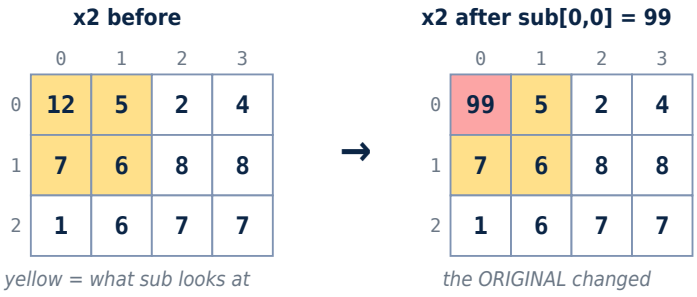
When you slice a NumPy array you do **not** get new data. You get a **view**: another way of looking at the **same** memory. So if you change the slice, the original array changes too.

PYTHON • type this in Colab

```
x2 = np.array([[12, 5, 2, 4],
               [ 7, 6, 8, 8],
               [ 1, 6, 7, 7]])
sub = x2[:2, :2]
print(sub)
sub[0, 0] = 99
print(sub)
print(x2)
```

▶ OUTPUT

```
[[12  5]
 [ 7  6]]
[[99  5]
 [ 7  6]]
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```



ANALOGY

View = a window onto a garden. If someone plants a flower in the garden, you see it through the window. If you could reach through the window and plant one, the garden changes.

Copy = a photograph of the garden. Drawing on the photo does not touch the garden.

COMMON TRAP

This is the opposite of Python lists. A list slice `L[:2]` is a **copy**. A NumPy slice is a **view**. If you carry your list instincts into NumPy, you will corrupt your data without any error.

7.2 Making a real copy: .copy()

PYTHON • type this in Colab

```
x2 = np.array([[12, 5, 2, 4],
               [ 7, 6, 8, 8],
               [ 1, 6, 7, 7]])
sub_copy = x2[:2, :2].copy()
sub_copy[0, 0] = 42
print(sub_copy)
print(x2)
```

▶ OUTPUT

```
[[42  5]
 [ 7  6]]
[[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

Now the original is untouched: `.copy()` allocated brand-new memory.

	View (plain slice)	Copy (.copy())
New memory used?	No	Yes
Changing it changes the original?	**Yes**	No
Speed on big data	Instant	Slower (data is copied)
Use when	You only read, or want to edit in place	You must protect the original

Testing it: np.shares_memory

PYTHON • type this in Colab

```
a = np.arange(5)
b = a[1:4]
cp = a[1:4].copy()
print(np.shares_memory(a, b))
print(np.shares_memory(a, cp))
```

▶ OUTPUT

```
True
False
```

WHY IT MATTERS

Why views exist: a dataset can be gigabytes. If every slice copied the data you would run out of memory. Views let you process pieces for free.

Why it bites: you take `X_train = X[:800]` and normalise it in place. The first 800 rows of the original `X` have silently changed too. The same trap hides in train/test splits, feature engineering and data cleaning. **Rule: if you will modify a slice and the original matters, add .copy().**

INTERVIEW FLAG

"What is the difference between a view and a copy in NumPy?"

A view shares the same memory buffer as the original, so changes to one show up in the other; slicing returns views. A copy has its own memory and is independent; you get one with .copy().

NOTE

Later you will learn **fancy indexing** and **boolean masks**. Unlike slices, those return **copies**. Just remember: *plain slices are views*.

✎ PRACTICE PROBLEMS | Part 7: Views vs Copies

Q1 **PREDICT THE OUTPUT**

After these lines, what are **a**, **b** and **c**?

PYTHON • type this in Colab

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]
c = a[1:4].copy()
b[0] = 100
c[1] = 200
print(a)
print(b)
print(c)
```

Q2 **FIND THE BUGS**

A student keeps the first 7 numbers as "training data" and zeroes the first value of it. The original **data** should stay safe, but it does not. Why? Fix it.

PYTHON • type this in Colab

```
data = np.arange(10)
train = data[:7]
train[0] = 99
print(data)
```

Q3 **PREDICT THE OUTPUT**

Predict both lines.

PYTHON • type this in Colab

```
a = np.arange(6)
print(np.shares_memory(a, a[2:5]))
print(np.shares_memory(a, a[2:5].copy()))
```

Q4 **THINK IT THROUGH**

Give one reason why NumPy returns views instead of copies, and one danger of that design.

Q5 **THINK IT THROUGH**

Python list: **L = [1, 2, 3, 4]; M = L[:2]; M[0] = 99**. Does L change? How is this different from NumPy?

Q6 **THEORY**

[5 marks] Explain view and copy in NumPy with an example. State how to create a copy and why this matters in machine learning.

PART 8

8 Reshaping and newaxis

Block 2: same numbers, different arrangement

IN THIS PART YOU WILL LEARN

- ✓ How reshape rearranges data without changing it
- ✓ The size rule, and the very useful -1 shortcut
- ✓ Row vectors and column vectors, and the shape (3,) vs (1, 3) vs (3, 1) puzzle
- ✓ Two ways to add a dimension: reshape and np.newaxis

8.1 reshape

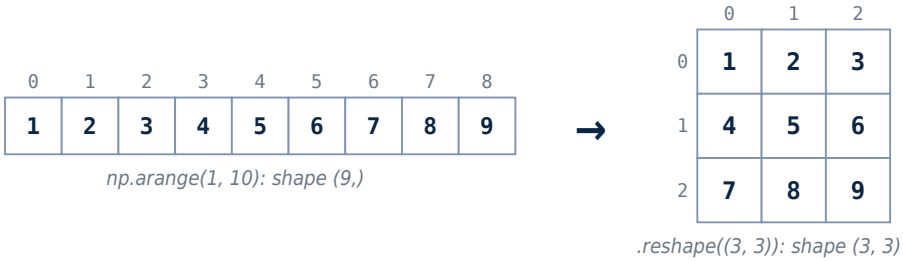
Reshaping changes how the same numbers are **arranged**, not the numbers themselves. The values keep their order and are simply re-cut into a new grid. **MUST KNOW**

PYTHON • type this in Colab

```
grid = np.arange(1, 10).reshape((3, 3))
print(grid)
```

▶ OUTPUT

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```



IMPORTANT **The size must match.** The old array and the new shape must hold the same number of items. 9 numbers fit 3x3, 1x9 or 9x1, but not 2x5.

PYTHON • type this in Colab

```
np.arange(10).reshape(3, 3)
```

▶ OUTPUT

```
ValueError: cannot reshape array of size 10 into shape (3,3)
```

The -1 shortcut

Put **-1** in one position and NumPy works out that number for you. This appears in almost all ML code. **MUST KNOW**

PYTHON • type this in Colab

```
x = np.arange(12)
print(x.reshape(3, -1).shape)
print(x.reshape(-1, 2).shape)
print(x.reshape(2, 2, -1).shape)
```

▶ OUTPUT

```
(3, 4)
(6, 2)
(2, 2, 3)
```

NOTE

You may use **-1 in only one position**. The book does not mention it, but it is standard practice. `x.reshape(-1, 1)` means "make me a column, whatever the length".

Reshape usually gives a view

PYTHON • type this in Colab

```
print(np.shares_memory(x, x.reshape(3, 4)))
```

▶ OUTPUT

```
True
```

As in the last topic, where possible reshape returns a **view** of the same memory, so changing the reshaped array can change the original.

8.2 Row vectors, column vectors and np.newaxis

A plain 1D array has shape `(3,)` and is neither a row nor a column. Linear algebra needs the difference, so we **add a dimension**. **MUST KNOW**

PYTHON • type this in Colab

```
x = np.array([1, 2, 3])
print(x.shape)
print(x.reshape((1, 3)))      # row vector, via reshape
print(x[np.newaxis, :])       # row vector, via newaxis
```

▶ OUTPUT

```
(3,)
[[1 2 3]]
[[1 2 3]]
```

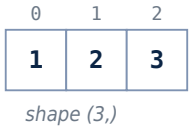
PYTHON • type this in Colab

```
print(x.reshape((3, 1)))      # column vector, via reshape
print(x[:, np.newaxis])       # column vector, via newaxis
print(x[:, np.newaxis].shape)
```

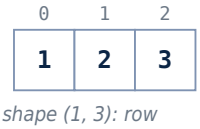
▶ OUTPUT

```
[[1]
 [2]
 [3]]
[[1]
 [2]
 [3]]
(3, 1)
```

Array	Shape	Looks like
`x`	`(3,)`	a flat line of 3 numbers (not a row, not a column)
`x.reshape(1, 3)` or `x[np.newaxis, :]`	`(1, 3)`	**row** vector: 1 row, 3 columns
`x.reshape(3, 1)` or `x[:, np.newaxis]`	`(3, 1)`	**column** vector: 3 rows, 1 column



shape (3,)



shape (1, 3): row



shape (3, 1): column

MEMORY TRICK

Read `np.newaxis` as "**insert a new dimension here**". Where you put it decides the result: `[np.newaxis, :]` (new axis in the *row* slot) makes a row; `[:, np.newaxis]` (new axis in the *column* slot) makes a column.

WHY IT MATTERS

In your Linear Algebra videos vectors are **columns**. In NumPy a flat array is neither, and mixing up (3,), (1, 3) and (3, 1) is a classic source of shape bugs. It also controls **broadcasting**, which is coming next in the NumPy chapter.

✎ PRACTICE PROBLEMS | Part 8: Reshaping and newaxis

Q1 BUILD IT

Turn **x = np.arange(1, 7)** into (a) a 2x3 array; (b) a column vector. Print the shape of each.

Q2 FIND THE BUGS

This crashes. Why? Name two shapes that would work.

PYTHON • type this in Colab

```
x = np.arange(10)
print(x.reshape(3, 4))
```

Q3 PREDICT THE OUTPUT

Predict the three shapes.

PYTHON • type this in Colab

```
x = np.arange(24)
print(x.reshape(4, -1).shape)
print(x.reshape(-1, 3).shape)
print(x.reshape(2, 3, -1).shape)
```

Q4 PREDICT THE OUTPUT

What are the three shapes?

PYTHON • type this in Colab

```
x = np.array([10, 20, 30, 40])
print(x.shape)
print(x[np.newaxis, :].shape)
print(x[:, np.newaxis].shape)
```

Q5 THINK IT THROUGH

Why is a 1D array of shape (3,) **not** the same thing as a column vector? Why does it matter?

Q6 **THINK IT THROUGH**

Does **reshape** change the numbers? If you change one value of the reshaped array, can the original change too?

Q7 **THEORY**

[3 marks] What is the use of -1 in reshape? What is the difference between **x.reshape(3,1)** and **x.reshape(1,3)**?

PART 9

9 Concatenation and Splitting

Block 2: joining arrays together and cutting them apart

IN THIS PART YOU WILL LEARN

- ✓ np.concatenate and the meaning of axis=0 and axis=1
- ✓ vstack, hstack and dstack as friendlier shortcuts
- ✓ The shape rule that makes joins fail
- ✓ np.split, vsplit and hsplit, and the "N cuts make N + 1 pieces" rule

9.1 np.concatenate: joining

Pass the arrays inside a **list** as the first argument. You can join two or many arrays. **MUST KNOW**

PYTHON • type this in Colab

```
x = np.array([1, 2, 3])
y = np.array([3, 2, 1])
print(np.concatenate([x, y]))
z = [99, 99, 99]
print(np.concatenate([x, y, z]))
```

▶ OUTPUT

```
[1 2 3 3 2 1]
[ 1  2  3  3  2  1 99 99 99]
```

Two dimensions and the axis

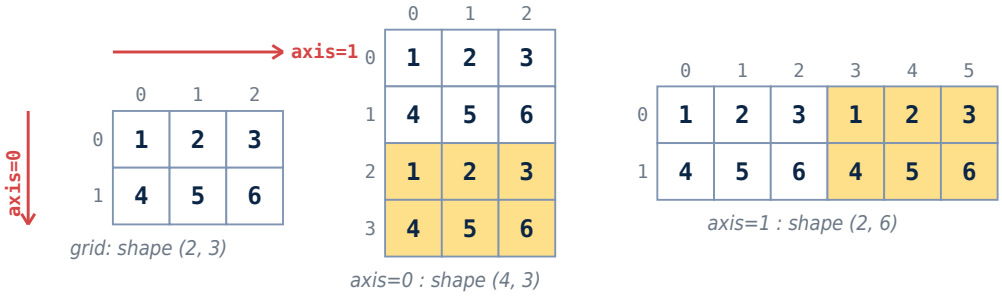
For tables you must say **which direction** to join. **axis=0** means "down the rows" (stack on top), **axis=1** means "across the columns" (stack side by side). The default is axis=0.

PYTHON • type this in Colab

```
grid = np.array([[1, 2, 3],
                 [4, 5, 6]])
print(np.concatenate([grid, grid]))
print(np.concatenate([grid, grid], axis=1))
```

▶ OUTPUT

```
[[1 2 3]
 [4 5 6]
 [1 2 3]
 [4 5 6]]
[[1 2 3 1 2 3]
 [4 5 6 4 5 6]]
```



MEMORY TRICK **axis=0 grows the number of rows** (the first number of the shape). **axis=1 grows the number of columns** (the second number). The axis number is the position in the shape tuple.

9.2 vstack, hstack and dstack

When arrays have different dimensions these are clearer than concatenate. **v**stack = vertical (adds rows), **h**stack = horizontal (adds columns). **MUST KNOW**

PYTHON • type this in Colab

```
x = np.array([1, 2, 3])
grid2 = np.array([[9, 8, 7],
                  [6, 5, 4]])
print(np.vstack([x, grid2]))
y = np.array([[99],
              [99]])
print(np.hstack([grid2, y]))
print(np.dstack([grid2, grid2]).shape)
```

▶ OUTPUT

```
[[1 2 3]
 [9 8 7]
 [6 5 4]]
[[ 9  8  7 99]
 [ 6  5  4 99]]
(2, 3, 2)
```

`np.dstack` stacks along the **third** axis (depth), so two (2, 3) arrays become one (2, 3, 2) array. You will rarely need it. **LOOK UP**

The rule that makes joins fail

IMPORTANT Arrays must match in **every dimension except the one you join along**.
vstack needs the same number of **columns**. **hstack** needs the same number of **rows**.

PYTHON • type this in Colab

```
a = np.array([[1, 2],
              [3, 4]])      # shape (2, 2)
b = np.array([[5, 6]])      # shape (1, 2)
print(np.vstack([a, b]))
print(np.hstack([a, b]))
```

▶ OUTPUT

```
[[1 2]
 [3 4]
 [5 6]]
ValueError: all the input array dimensions except for the concatenation axis must match exactly,
     but along dimension 0, the array at index 0 has size 2 and the array at index 1 has size 1
```

The vstack worked (both have 2 columns), so the 3x2 result printed first. The hstack tried to put a 1-row array beside a 2-row array and failed.

WHY IT MATTERS `hstack` adds a new **feature column** to a dataset. `vstack` appends new **samples** (rows). Splitting is the idea underneath a train/test split.

9.3 np.split, vsplit and hsplit

Splitting is the opposite of joining. You give the **split points** (the indexes where to cut), not the size of the pieces.

PYTHON • type this in Colab

```
x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)
```

▶ OUTPUT

```
[1 2 3] [99 99] [3 2 1]
```

MEMORY TRICK

N split points make N + 1 pieces. Two cuts [3, 5] make three pieces: indexes 0-2, then 3-4, then 5 to the end. Cuts are made *before* the given indexes.

PYTHON • type this in Colab

```
grid = np.arange(16).reshape((4, 4))
upper, lower = np.vsplit(grid, [2])
print(upper)
print(lower)
left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

▶ OUTPUT

```
[[0 1 2 3]
 [4 5 6 7]]
[[ 8  9 10 11]
 [12 13 14 15]]
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

vsplit(grid, [2]): rows 0-1 | rows 2-3

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

hsplit(grid, [2]): cols 0-1 | cols 2-3

NOTE

`np.dsplit` splits along the third axis. A handy extra: `np.split(a, 3)` with a *single number* cuts into that many equal pieces (the length must divide evenly).

PYTHON • type this in Colab

```
print(np.split(np.arange(9), 3))
```

▶ OUTPUT

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

✎ PRACTICE PROBLEMS | Part 9: Concatenation and Splitting

Q1 PREDICT THE OUTPUT

Predict the shape of each result.

PYTHON • type this in Colab

```
a = np.ones((2, 3))
b = np.ones((4, 3))
c = np.ones((2, 5))
print(np.vstack([a, b]).shape)
print(np.hstack([a, c]).shape)
print(np.concatenate([a, a], axis=1).shape)
```

Q2 PREDICT THE OUTPUT

Predict the three pieces. Remember that cuts happen **before** the given indexes.

PYTHON • type this in Colab

```
print(np.split(np.arange(10), [3, 7]))
```

Q3 FIND THE BUGS

One of the two lines crashes. Which one, and why?

PYTHON • type this in Colab

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
print(np.vstack([a, b]).shape)
print(np.hstack([a, b]).shape)
```

Q4 BUILD IT

You have a feature table **X** of shape (4, 3) and one more feature column **f** (4 values). Add the column to X so the result has shape (4, 4).

Q5 BUILD IT

Split **np.arange(10)** into a "train" part of the first 8 values and a "test" part of the last 2 using **np.split**.

Q6 THINK IT THROUGH

What does **axis=0** mean and what does **axis=1** mean in **np.concatenate**? How can you remember it?

Q7 THINK IT THROUGH

You split a 4x4 grid with **np.hsplit(grid, [2])**. What are the shapes of the two pieces?

Q8 THEORY

[4 marks] State the rule that decides whether two arrays can be joined with vstack or hstack. Give one example that fails.

PART 10

10 Revision Zone

Cheat sheets, interview quick-fire, error radar and a final mixed paper

IN THIS PART YOU WILL LEARN

- ✓ One-page function cheat sheet with priorities
- ✓ Topic checklist for Blocks 1 and 2
- ✓ 20 interview questions with model answers
- ✓ The error radar: mistakes to check before you trust your output
- ✓ A final mixed paper covering everything

10.1 Function cheat sheet

Group	Function / attribute	What it does	Example	Priority
Create	`np.array`	list to array	`np.array([1, 2, 3])`	MUST KNOW
	`np.zeros / np.ones`	filled with 0 / 1	`np.zeros((2, 3))`	MUST KNOW
	`np.full`	filled with a value	`np.full((2, 3), 7)`	GOOD TO KNOW
	`np.arange`	step sequence, stop excluded	`np.arange(0, 10, 2)`	MUST KNOW
	`np.linspace`	N points, stop included	`np.linspace(0, 1, 5)`	MUST KNOW
	`np.eye`	identity matrix	`np.eye(3)`	MUST KNOW
	`np.empty`	junk values (avoid)	`np.empty(3)`	LOOK UP
Random	`np.random.seed`	repeatable randomness	`np.random.seed(0)`	MUST KNOW
	`np.random.random`	uniform 0 to 1	`np.random.random((2, 3))`	GOOD TO KNOW
	`np.random.normal`	bell curve	`np.random.normal(0, 1, (3, 3))`	MUST KNOW
	`np.random.randint`	whole numbers, high excluded	`np.random.randint(0, 10, 5)`	GOOD TO KNOW
Type	`dtype=`	choose the type	`np.array(a, dtype='float32')`	MUST KNOW
	`.astype`	convert type	`x.astype(int)`	MUST KNOW
	`np.iinfo / np.finfo`	limits of a type	`np.iinfo(np.uint8).max`	LOOK UP
Attributes	`.shape`	size of each dimension	`x.shape`	MUST KNOW
	`.ndim .size`	dimensions / total items	`x.ndim`	GOOD TO KNOW
	`.dtype`	element type	`x.dtype`	MUST KNOW
	`.itemsize .nbytes`	bytes per item / total	`x.nbytes`	GOOD TO KNOW
Access	`x[i] x[r, c]`	one element	`x2[2, -1]`	MUST KNOW
	`x[a:b:s]`	slice (a view)	`x[::-1]`	MUST KNOW
	`x[:, j] x[i, :]`	column / row	`x2[:, 0]`	MUST KNOW
	`.copy()`	independent copy	`x[:3].copy()`	MUST KNOW
	`np.shares_memory`	view test	`np.shares_memory(a, b)`	LOOK UP
Reshape	`.reshape`	new arrangement	`x.reshape(3, -1)`	MUST KNOW

Group	Function / attribute	What it does	Example	Priority
	`np.newaxis`	add a dimension	`x[:, np.newaxis]`	MUST KNOW
Join/split	`np.concatenate`	join along an axis	`np.concatenate([a, b], axis=1)`	MUST KNOW
	`np.vstack / np.hstack`	join vertically / horizontally	`np.hstack([a, b])`	MUST KNOW
	`np.dstack`	join along depth	`np.dstack([a, b])`	LOOK UP
	`np.split / vsplit / hsplit`	cut apart	`np.split(x, [3, 5])`	GOOD TO KNOW

10.2 Topic checklist

Topic from the blocks	Covered in	Done
Block 1: Python objects and the heavy list	Part 1	✓
Block 1: Fixed-type arrays and the array module	Part 1	✓
Block 1: np.array, upcasting, dtype, multi-dimensional lists	Part 2, 2.1 to 2.4	✓
Block 1: Arrays from scratch (zeros, ones, full, arange, linspace, random, eye, empty)	Part 2, 2.5	✓
Block 1: Standard NumPy data types	Part 3	✓
Block 2: Attributes (ndim, shape, size, dtype, itemsize, nbytes)	Part 4	✓
Block 2: Indexing (single elements)	Part 5	✓
Block 2: Slicing (1D, 2D, rows and columns)	Part 6	✓
Block 2: Views and copies	Part 7	✓
Block 2: Reshaping and newaxis	Part 8	✓
Block 2: Concatenation and splitting	Part 9	✓

10.3 Interview quick-fire: 20 questions with model answers

#	Question	Model answer
1	Why is NumPy faster than a list?	Fixed-type values in one contiguous block and operations run as compiled C loops (vectorization).
2	What does ndarray stand for?	N-dimensional array.
3	What is vectorization?	Applying an operation to a whole array at once without a Python loop.
4	What does [1, 2] * 2 give for a list and for an array?	List: [1, 2, 1, 2] (repeat). Array: [2, 4] (element-wise).
5	What is upcasting?	NumPy converting mixed values to the strongest common type: bool, then int, then float, then string.
6	Default dtype of np.array([1, 2, 3]) and np.array([1., 2.])?	int64 and float64 on most modern systems.
7	arange vs linspace?	arange takes a step and excludes the stop; linspace takes a count and includes the stop.
8	np.random.randint(0, 10) can it return 10?	No. The high value is excluded: results are 0 to 9.
9	Why set a random seed?	For reproducible results: the same seed gives the same random numbers.

#	Question	Model answer
10	What is float32 vs float64?	32-bit (4 bytes) vs 64-bit (8 bytes) decimals. Deep learning defaults to float32.
11	What is uint8 used for?	Image pixels: unsigned 8-bit integers from 0 to 255.
12	What is overflow?	A result larger than the dtype maximum wraps around silently (uint8: 200 + 100 = 44).
13	shape vs size vs ndim?	Tuple of dimension lengths; total number of items; number of dimensions.
14	nbytes formula?	nbytes = itemsize x size.
15	What does x[:, 0] return for a 2D array?	The first column (all rows, column 0), as a 1D array.
16	How do you reverse an array?	x[::-1].
17	View vs copy?	A view shares memory with the original (slices); a copy has its own memory (.copy()).
18	What does reshape(3, -1) do?	Makes 3 rows and lets NumPy compute the number of columns from the size.
19	How do you turn a 1D array into a column vector?	x.reshape(-1, 1) or x[:, np.newaxis], giving shape (n, 1).
20	What does axis=1 mean in concatenate?	Join along the second dimension: columns grow, arrays go side by side.

10.4 The error radar: check these before you trust your output

#	Mistake	How to spot / fix it
1	Shape given as two arguments: np.zeros(3, 4)	Shape is ONE tuple: np.zeros((3, 4))
2	Thinking arange includes the stop	It excludes it. arange(1, 10, 3) is [1 4 7]
3	Mixing up arange and linspace arguments	arange: step. linspace: count (stop included)
4	np.random.randint(1, 6) for a dice	High is excluded: use (1, 7)
5	A stray string in the data	The whole array becomes text. Check .dtype
6	Putting decimals into an int array	Silent truncation. Create the array as float
7	uint8 overflow	Convert with astype to a bigger type first
8	Assuming the dtype when counting bytes	Print .dtype, then use nbytes = itemsize x size
9	x.shape() with brackets	shape is an attribute. No brackets
10	x[::2] vs x[1::2]	Missing start means 0
11	Modifying a slice and forgetting it is a view	Add .copy() if the original matters
12	Reshape with the wrong total size	Old size must equal new size. Use -1 to let NumPy calculate
13	hstack/vstack with mismatched dimensions	Match rows (hstack) or columns (vstack)
14	Treating split points as piece sizes	They are cut indexes: N cuts give N + 1 pieces
15	Using np.empty and reading garbage	Use np.zeros unless you overwrite everything
16	Expecting array module to do maths	It only stores. NumPy does the operations

✎ PRACTICE PROBLEMS | Final Mixed Paper (Blocks 1 and 2)

Q1 PREDICT THE OUTPUT

Predict every printed value. (Numbers are repeatable because of the seed.)

PYTHON • type this in Colab

```python
np.random.seed(0)
x = np.random.randint(10, size=(3, 4))
print(x)
print(x[1, -1])
print(x[:, 0])
print(x[::-1, 0])
```

Q2 FIND THE BUGS

data should stay unchanged but does not. Explain and fix.

PYTHON • type this in Colab

```python
data = np.arange(10)
train = data[:7]
train[0] = 99
print(data)
```

Q3 BUILD IT

Create the numbers 1 to 12 as a 3x4 array. Then print (a) its second column, (b) its last row reversed, (c) its shape.

Q4 CALCULATE

Give the shape of each result: (a) vstack of (2,3) and (4,3); (b) hstack of (2,3) and (2,5); (c) np.dstack of two (2,3) arrays.

Q5 PREDICT THE OUTPUT

What does this print?

PYTHON • type this in Colab

```python
print(np.split(np.arange(10), [3, 7]))
```

Q6 BUILD IT

Turn **x = np.array([1, 2, 3])** into a column vector in three different ways and print the shape.

Q7 CALCULATE

A batch holds 32 colour images of 64 x 64 pixels (3 channels) as float32. Give the shape, size and nbytes.

Q8 THEORY

[5 marks] Explain with examples: (i) why NumPy is faster than a list, (ii) arange vs linspace, (iii) view vs copy.

ANSWER KEY

HOW TO USE

HOW TO USE. Try every question yourself first. Then compare. If your answer is different, do not just copy mine: find *why* the difference exists. That is where the learning happens.

Every output shown below was produced by really running the code.

Answers: Part 1: Why NumPy?

Q1 THEORY

1. Every list item is a full Python object with its own type information and reference count, and the list stores only pointers to them. A NumPy array stores just the raw values.

2. A list may hold mixed types, so type information must be repeated for every item. A NumPy array stores one type for the whole block.

Contiguous means the values sit next to each other in memory with no gaps, like eggs in a tray. This makes reading and operating on them fast.

Q2 PREDICT THE OUTPUT

▶ **OUTPUT**

```
['bool', 'str', 'float', 'int']
```

A list happily stores four different types because each item is its own full object that knows its type.

Q3 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[1, 2, 3, 1, 2, 3, 1, 2, 3]
[3 6 9]
```

On a list, * means repetition (copies are joined). On a NumPy array it is element-wise multiplication. This is vectorization.

Q4 PREDICT THE OUTPUT

▶ **OUTPUT**

```
[1, 2, 3, 4]
[4 6]
```

List + joins two lists. Array + adds element by element: 1+3 and 2+4.

Q5 THINK IT THROUGH

A NumPy array.

Memory: a list keeps a pointer plus a full object (type, reference count, value) for each of the 5 million items; an array keeps only the raw numbers, packed together.

Speed: A * 2 runs as one loop in compiled C over a contiguous block; a list needs a Python-level loop that follows a pointer to every object.

Q6 THINK IT THROUGH

SOLUTION CODE

```
import array
print(array.array('i', [1, 2, 3]) * 2)
```

▶ **OUTPUT**

```
array('i', [1, 2, 3, 1, 2, 3])
```

The array module only provides compact **storage** and it is 1D only. It cannot do element-wise maths: * repeats the array like a list. NumPy gives compact storage **plus** fast vectorized operations and multi-dimensional arrays.

Q7 CALCULATE

SOLUTION CODE

```
print(np.zeros(1_000_000, dtype="int64").nbytes)
print(np.zeros(1_000_000, dtype="float32").nbytes)
```

▶ OUTPUT

```
8000000
4000000
```

(a) 1,000,000 x 8 = 8,000,000 bytes. (b) 1,000,000 x 4 = 4,000,000 bytes. Halving the item size halves the memory.

Answers: Part 2: Creating Arrays

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[1.  2.5 1. ]
float64
```

Mixed bool, int and float. The strongest type is float, so every value is converted to float64: True becomes 1.0 and 1 becomes 1.0. The ladder is bool < int < float < string.

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
['1' 'a' '2.5']
<U32
```

One string pulls the whole array up to text (Unicode strings), so even the numbers become the strings '1' and '2.5'. Text cannot be added or multiplied like numbers.

Q3 BUILD IT

SOLUTION CODE

```
x = np.array([[1, 2, 3], [4, 5, 6]], dtype="float32")
print(x)
print(x.dtype)
```

▶ OUTPUT

```
[[1. 2. 3.]
 [4. 5. 6.]]
float32
```

Nested lists make rows; dtype= sets the type.

Q4 PREDICT THE OUTPUT

▶ OUTPUT

```
[1 4 7]
[ 1.  4.  7. 10.]
```

arange starts at 1 and adds the step 3: 1, 4, 7, then 10 is the stop and is excluded. **linspace** gives exactly 4 points from 1 to 10 with the stop included, so the step works out to 3.

Q5 CALCULATE

▶ OUTPUT

```
[0 2 4 6 8] 5
[ 0.  2.  4.  6.  8. 10.] 6
```

(a) 5 numbers, last is 8 (10 is excluded). (b) 6 numbers, last is 10.0 (stop included).

Q6 BUILD IT

SOLUTION CODE

```
np.random.seed(0)
print(np.random.normal(0, 1, (4, 4)))
```

▶ OUTPUT

```
[[ 1.76405235  0.40015721  0.97873798  2.2408932 ]
 [ 1.86755799 -0.97727788  0.95008842 -0.15135721]
 [-0.10321885  0.4105985   0.14404357  1.45427351]
 [ 0.76103773  0.12167502  0.44386323  0.33367433]]
```

Shape goes in as a tuple (4, 4). The seed makes the numbers identical on every run.

Q7 FIND THE BUGS

(a) Shape must be a single tuple. `np.zeros(3, 4)` passes 4 as the dtype and fails with *TypeError: Cannot interpret '4' as a data type*. Fix: `np.zeros((3, 4))`.

(b) The third argument of linspace is the **count**, which must be a whole number; 0.25 fails with *TypeError: 'float' object cannot be interpreted as an integer*. To get 0.25 steps use `np.arange(0, 1.25, 0.25)` or `np.linspace(0, 1, 5)`.

(c) The high value is excluded, so 6 can never appear. Fix: `np.random.randint(1, 7, 10)`.

Q8 BUILD IT

SOLUTION CODE

```
print(np.eye(3, dtype=int))
print(np.linspace(-1, 1, 10))
print(np.full((2, 5), 7))
```

▶ OUTPUT

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
[-1.         -0.77777778 -0.55555556 -0.33333333 -0.11111111  0.11111111
  0.33333333  0.55555556  0.77777778  1.        ]
[[7 7 7 7 7]
 [7 7 7 7 7]]
```

eye for the identity, linspace for exact count and exact ends, full for a constant fill.

Q9 THINK IT THROUGH

It allocates memory without writing anything into it, so the array contains leftover junk that looks like valid numbers. If you read a slot you never filled, you get garbage with no error. Use **np.zeros** (or np.full) so every slot has a known value.

Q10 THEORY

1. arange takes a step size, linspace takes a count of points. **2.** arange excludes the stop value, linspace includes it (and always returns floats).

Seed: random numbers come from a formula that starts at the seed. A fixed seed gives the same numbers every run, so experiments are reproducible and results can be compared fairly.

Answers: Part 3: Data Types

Q1 READ & DECODE

(a) decimal numbers stored in 32 bits = 4 bytes each. (b) unsigned whole numbers (no negatives) in 8 bits = 1 byte each, range 0 to 255. (c) signed whole numbers in 16 bits = 2 bytes each, range -32,768 to 32,767. (d) complex numbers made of two 64-bit floats = 16 bytes each.

Q2 CALCULATE

SOLUTION CODE

```
print(np.iinfo(np.int8).min, np.iinfo(np.int8).max)
print(np.iinfo(np.uint16).max)
```

▶ OUTPUT

```
-128 127
65535
```

(a) 8 bits give 2^8 = 256 values, from -128 to 127. (b) 16 bits unsigned: 2^16 - 1 = 65,535.

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
[4]
```

The true result 260 does not fit in uint8 (maximum 255). It wraps around: 260 - 256 = 4.

Q4 CALCULATE

SOLUTION CODE

```
n = 224 * 224 * 3
print(n)
print(np.zeros((224, 224, 3), dtype=np.uint8).nbytes)
print(np.zeros((224, 224, 3), dtype=np.float32).nbytes)
```

▶ OUTPUT

```
150528
150528
602112
```

Values: 224 x 224 x 3 = 150,528. As uint8 (1 byte each) = 150,528 bytes. As float32 (4 bytes each) = 602,112 bytes. Converting the image to float32 makes it 4 times bigger.

Q5 THINK IT THROUGH

1. Memory: float32 uses 4 bytes per number, float64 uses 8, so a model and its data take half the space. **2. Speed:** smaller numbers are faster to move and compute, especially on GPUs, and neural networks do not need the extra precision.

Q6 PREDICT THE OUTPUT

▶ OUTPUT

```
[ 1  2 -1]
```

No rounding. It chops the decimal part, moving towards zero: 1.7 becomes 1, 2.9 becomes 2, -1.7 becomes -1.

Q7 FIND THE BUGS

▶ OUTPUT

```
[150 250  44]
```

SOLUTION CODE

```
pixels = np.array([100, 200, 250], dtype=np.uint8)
safe = pixels.astype(np.int16) + 50
print(safe)
print(np.clip(safe, 0, 255))
```

▶ OUTPUT

```
[150 250 300]
[150 250 255]
```

uint8 cannot hold 300 (maximum 255), so it overflowed silently: 300 - 256 = 44. Fix: convert to a bigger type (int16) before adding. Bonus: **np.clip(safe, 0, 255)** then caps the values at the valid pixel range.

Q8 THEORY

Overflow happens when a result is larger than the maximum of the array's dtype. NumPy does not raise an error; the value wraps around. Example: a uint8 array holding 200 plus 100 gives 44, not 300, which silently corrupts image data. To avoid it, convert to a larger dtype with **astype** before calculating (or clip the result), and always check **.dtype**.

Answers: Part 4: Array Attributes

Q1 READ & DECODE

SOLUTION CODE

```
x = np.zeros((5, 3, 2))
print(x.ndim, x.size)
```

▶ OUTPUT

```
3 30
```

ndim = 3 (three numbers in the shape). **size = 5 x 3 x 2 = 30**. In words: 5 layers, each with 3 rows and 2 columns.

Q2 CALCULATE

SOLUTION CODE

```
x = np.zeros((5, 3, 2), dtype=np.int32)
print(x.itemsize, x.nbytes)
```

▶ OUTPUT

```
4 120
```

int32 is 32 bits = 4 bytes, so itemsize = 4. nbytes = itemsize x size = 4 x 30 = 120.

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
2 (4, 6) 24 4 96
```

ndim 2 (two dimensions), shape (4, 6), size 4 x 6 = 24, itemsize 4 bytes (float32), nbytes 24 x 4 = 96.

Q4 CALCULATE

SOLUTION CODE

```
data = np.zeros((100, 28, 28), dtype=np.uint8)
print(data.shape, data.size, data.nbytes)
```

▶ OUTPUT

```
(100, 28, 28) 78400 78400
```

Shape (100, 28, 28). Size = 100 x 28 x 28 = 78,400. uint8 uses 1 byte each, so nbytes = 78,400.

Q5 FIND THE BUGS

▶ OUTPUT

```
TypeError: 'tuple' object is not callable
```

shape is an attribute, not a method, so it takes no brackets. The tuple **(12,)** was returned and then Python tried to "call" it. Fix: **print(x.shape)**.

Q6 THINK IT THROUGH

Shapes are tuples. In Python a one-item tuple needs a trailing comma, because (6) is just the number 6 in brackets. So (6,) is the way to write a tuple with one number.

Q7 CALCULATE

SOLUTION CODE

```
x = np.ones((10, 10))
print(x.size, x.itemsize, x.nbytes)
print(x.itemsize * x.size == x.nbytes)
```

▶ OUTPUT

```
100 8 800
True
```

size 100, itemsize 8 (float64), nbytes 800 = 8 x 100.

Q8 THEORY

shape: tuple giving the length of each dimension, e.g. (3, 4). **ndim**: number of dimensions, e.g. 2. **size**: total elements, e.g. 12. **dtype**: type of every element, e.g. float64. Printing the shape catches shape-mismatch errors before they crash an operation or silently produce wrong results.

Answers: Part 5: Indexing

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
70 120 30
```

x[1, 2]: row 1, column 2 = 70. x[-1, -1]: last row, last column = 120. x[0, -2]: row 0, second from the end = 30.

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
7
```

7. The array is of integer type, so the decimal part is chopped (towards zero). It does not round to 8.

Q3 FIND THE BUGS

▶ OUTPUT

```
IndexError: index 6 is out of bounds for axis 0 with size 6
```

The array has 6 items, so valid indexes are 0 to 5 (or -6 to -1). Index 6 is out of bounds, hence IndexError. The last item is x[5] or x[-1].

Q4 BUILD IT

SOLUTION CODE

```
x = np.arange(1, 13).reshape(3, 4)
print(x)
print(x[-1, -1])
print(x[1, 0])
```

▶ OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
12
5
```

12 is in the last row and last column: x[-1, -1]. 5 is in row 1, column 0: x[1, 0].

Q5 THINK IT THROUGH

SOLUTION CODE

```
p = np.zeros(3, dtype=int)
p[0], p[1], p[2] = 0.87, 0.12, 0.99
print(p)
```

▶ OUTPUT

```
[0 0 0]
```

All zeros, because each decimal is truncated to its integer part with no warning. Create the array as float (np.zeros(3) or dtype=float) so decimals are kept.

Answers: Part 6: Slicing

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[2 3 4 5]
[0 3 6 9]
[7 8 9]
[9 7 5 3 1]
```

x[2:6] takes indexes 2 to 5. x[::3] takes every third item from 0. x[-3:] takes the last three. x[::-2] reverses and takes every other item starting from the end.

Q2 BUILD IT

SOLUTION CODE

```
x = np.arange(1, 21).reshape(4, 5)
print(x[:, -1])
print(x[:2, 3:])
print(x[:, ::2])
```

▶ OUTPUT

```
[ 5 10 15 20]
[[ 4  5]
 [ 9 10]]
[[ 1  3  5]
 [ 6  8 10]
 [11 13 15]
 [16 18 20]]
```

(a) **x[:, -1]**. (b) **x[:2, 3:]**: first two rows, columns from index 3. (c) **x[:, ::2]**: every other column starting at column 0. Careful: **x[:, 1::2]** would give columns 1 and 3 instead.

Q3 FIND THE BUGS

▶ OUTPUT

```
[[ 2  4]
 [ 7  9]
 [12 14]
 [17 19]]
```

1::2 starts at index 1, so it returns columns 1 and 3. A missing start means 0, so the correct slice is **x[:, ::2]**. Lesson: always compare the output with what you wanted, not just check that the code ran.

Q4 READ & DECODE

(a) the whole array reversed. (b) every second item starting from index 1. (c) the first column of a 2D array (all rows, column 0). (d) the last three items.

Q5 PREDICT THE OUTPUT

▶ OUTPUT

```
(4,)
(3,)
(2, 2)
```

A single row or a single column becomes **1D** with shape (4,) and (3,). Slicing both dimensions keeps 2D: (2, 2).

Q6 THINK IT THROUGH

SOLUTION CODE

```
x = np.arange(10)
print(x[5:100])
```

▶ OUTPUT

```
[5 6 7 8 9]
```

It returns [5 6 7 8 9]. Slices are clipped to the array's boundaries instead of failing, whereas a single index (x[100]) must exist and would raise an IndexError.

Q7 BUILD IT

SOLUTION CODE

```
x = np.arange(10)
print(x[7:])
print(x[::-2])
print(x[1:-1])
```

▶ OUTPUT

```
[7 8 9]
[9 7 5 3 1]
[1 2 3 4 5 6 7 8]
```

(a) x[7:] or x[-3:]. (b) x[::-2] starts at the end (9) and steps back by 2. (c) x[1:-1] starts at 1 and stops before the last item.

Q8 THEORY

Pattern: **x[start:stop:step]**. Defaults: start = 0, stop = end of the array, step = 1; start is included, stop is excluded. With a negative step the defaults for start and stop swap (start becomes the end, stop the beginning), so x[::-1] reverses the array.

Answers: Part 7: Views vs Copies

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[  1 100   3   4   5]
[100   3   4]
[  2 200   4]
```

b is a view, so b[0] = 100 changes a as well: a = [1 100 3 4 5]. c is an independent copy, so c[1] = 200 changes only c.

Q2 FIND THE BUGS

▶ OUTPUT

```
[99  1  2  3  4  5  6  7  8  9]
```

SOLUTION CODE

```
data = np.arange(10)
train = data[:7].copy()
train[0] = 99
print(data)
print(train)
```

▶ OUTPUT

```
[0 1 2 3 4 5 6 7 8 9]
[99  1  2  3  4  5  6]
```

data[:7] is a view, so editing train edits data. Fix: **data[:7].copy()** gives train its own memory.

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
True
False
```

A slice shares memory with its source (True). A copy does not (False).

Q4 THINK IT THROUGH

Reason: big arrays would eat memory and time if every slice copied the data; views are instant and free. **Danger:** modifying a slice silently changes the original array, which can corrupt data without any error.

Q5 THINK IT THROUGH

SOLUTION CODE

```
L = [1, 2, 3, 4]
M = L[:2]
M[0] = 99
print(L)
```

▶ OUTPUT

```
[1, 2, 3, 4]
```

L does not change: a list slice is a copy. A NumPy slice is a view, so the equivalent code on an array would change the original.

Q6 THEORY

A **view** is a new array object that looks at the same memory as the original; slicing returns views, so changing the view changes the original (e.g. sub = x2[:2, :2]; sub[0, 0] = 99 also changes x2). A **copy** owns separate memory; create it with .copy(). In ML, slicing a dataset into train and test parts and modifying one (for example normalising it) can silently alter the original data if a view was used, so copy before modifying.

Answers: Part 8: Reshaping and newaxis

Q1 BUILD IT

SOLUTION CODE

```
x = np.arange(1, 7)
a = x.reshape(2, 3)
b = x.reshape(-1, 1)
print(a)
print(a.shape)
print(b)
print(b.shape)
```

▶ OUTPUT

```
[[1 2 3]
 [4 5 6]]
(2, 3)
[[1]
 [2]
 [3]
 [4]
 [5]
 [6]]
(6, 1)
```

(a) reshape(2, 3). (b) reshape(6, 1), or reshape(-1, 1), or x[:, np.newaxis]. The column vector has shape (6, 1).

Q2 FIND THE BUGS

▶ OUTPUT

```
ValueError: cannot reshape array of size 10 into shape (3,4)
```

10 numbers cannot fill 12 slots (3 x 4). The sizes must match. Shapes that work: (2, 5), (5, 2), (1, 10), (10, 1).

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
(4, 6)
(8, 3)
(2, 3, 4)
```

-1 means "work it out". 24 / 4 = 6 columns; 24 / 3 = 8 rows; 24 / (2 x 3) = 4 for the last dimension.

Q4 PREDICT THE OUTPUT

▶ OUTPUT

```
(4,)
(1, 4)
(4, 1)
```

Flat: (4,). New axis in the row slot: (1, 4), a row. New axis in the column slot: (4, 1), a column.

Q5 THINK IT THROUGH

A 1D array has only one axis, so it has no row/column orientation. A column vector has two axes with shape (3, 1). Operations such as matrix multiplication and broadcasting treat (3,), (1, 3) and (3, 1) differently, so you must add the dimension yourself when orientation matters.

Q6 THINK IT THROUGH

SOLUTION CODE

```
x = np.arange(6)
y = x.reshape(2, 3)
y[0, 0] = 99
print(x)
```

▶ OUTPUT

```
[99  1  2  3  4  5]
```

Reshape never changes the values, only their arrangement. It normally returns a view, so changing y also changed x.

Q7 THEORY

-1 lets NumPy calculate that dimension from the total size (allowed in only one position). reshape(3, 1) gives a column vector (3 rows, 1 column); reshape(1, 3) gives a row vector (1 row, 3 columns).

Answers: Part 9: Concatenation and Splitting

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
(6, 3)
(2, 8)
(2, 6)
```

vstack adds rows: 2 + 4 = 6 rows, columns stay 3. hstack adds columns: 3 + 5 = 8, rows stay 2. concatenate along axis=1 doubles the columns: (2, 6).

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
[array([0, 1, 2]), array([3, 4, 5, 6]), array([7, 8, 9])]
```

Cuts before index 3 and before index 7 give indexes 0-2, 3-6 and 7-9. Two split points make three pieces.

Q3 FIND THE BUGS

▶ OUTPUT

```
(3, 2)
ValueError: all the input array dimensions except for the concatenation axis must match exactly,
     but along dimension 0, the array at index 0 has size 2 and the array at index 1 has size 1
```

The hstack line crashes. hstack needs the same number of rows, but a has 2 and b has 1. vstack works because both have 2 columns, giving shape (3, 2).

Q4 BUILD IT

SOLUTION CODE

```
X = np.zeros((4, 3))
f = np.array([1, 2, 3, 4])
new_X = np.hstack([X, f.reshape(-1, 1)])
print(new_X.shape)
```

▶ OUTPUT

```
(4, 4)
```

f is 1D, so first turn it into a column with reshape(-1, 1) (or f[:, np.newaxis]) giving shape (4, 1). Then hstack adds it as the last column.

Q5 BUILD IT

SOLUTION CODE

```
train, test = np.split(np.arange(10), [8])
print(train)
print(test)
```

▶ OUTPUT

```
[0 1 2 3 4 5 6 7]
[8 9]
```

One cut before index 8 gives two pieces: indexes 0-7 and 8-9.

Q6 THINK IT THROUGH

axis=0 joins along the first dimension (rows grow: stacking on top). axis=1 joins along the second dimension (columns grow: side by side). The axis number is the position in the shape tuple (rows, columns).

Q7 THINK IT THROUGH

SOLUTION CODE

```
grid = np.arange(16).reshape(4, 4)
left, right = np.hsplit(grid, [2])
print(left.shape, right.shape)
```

▶ OUTPUT

```
(4, 2) (4, 2)
```

hsplit cuts the columns before column 2, so left holds columns 0-1 and right holds columns 2-3: both are (4, 2).

Q8 THEORY

Arrays must have the same size in every dimension except the one being joined. vstack needs equal numbers of columns; hstack needs equal numbers of rows. Example: np.hstack of a (2, 2) array and a (1, 2) array fails because the row counts (2 and 1) differ.

Answers: Final Mixed Paper (Blocks 1 and 2)

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[[5 0 3 3]
 [7 9 3 5]
 [2 4 7 6]]
5
[5 7 2]
[2 7 5]
```

x[1, -1] is row 1, last column. x[:, 0] is the first column as 1D. x[::-1, 0] is the first column with the rows reversed.

Q2 FIND THE BUGS

▶ OUTPUT

```
[99  1  2  3  4  5  6  7  8  9]
```

SOLUTION CODE

```
data = np.arange(10)
train = data[:7].copy()
train[0] = 99
print(data)
```

▶ OUTPUT

```
[0 1 2 3 4 5 6 7 8 9]
```

A plain slice is a view, so train shares memory with data. Use .copy().

Q3 BUILD IT

SOLUTION CODE

```
x = np.arange(1, 13).reshape(3, 4)
print(x[:, 1])
print(x[-1, ::-1])
print(x.shape)
```

▶ OUTPUT

```
[ 2  6 10]
[12 11 10  9]
(3, 4)
```

arange(1, 13) holds 12 numbers, reshaped to (3, 4). Second column: x[:, 1]. Last row reversed: x[-1, ::-1].

Q4 CALCULATE

SOLUTION CODE

```
a = np.vstack([np.ones((2, 3)), np.ones((4, 3))]).shape
b = np.hstack([np.ones((2, 3)), np.ones((2, 5))]).shape
c = np.dstack([np.ones((2, 3)), np.ones((2, 3))]).shape
print(a, b, c)
```

▶ OUTPUT

```
(6, 3) (2, 8) (2, 3, 2)
```

(a) (6, 3), rows add. (b) (2, 8), columns add. (c) (2, 3, 2), the new third axis has length 2.

Q5 PREDICT THE OUTPUT

▶ OUTPUT

```
[array([0, 1, 2]), array([3, 4, 5, 6]), array([7, 8, 9])]
```

Cuts before indexes 3 and 7: three pieces.

Q6 BUILD IT

SOLUTION CODE

```
x = np.array([1, 2, 3])
print(x.reshape(3, 1).shape)
print(x.reshape(-1, 1).shape)
print(x[:, np.newaxis].shape)
```

▶ OUTPUT

```
(3, 1)
(3, 1)
(3, 1)
```

reshape(3, 1), reshape(-1, 1) and x[:, np.newaxis] all give shape (3, 1).

Q7 CALCULATE

SOLUTION CODE

```
b = np.zeros((32, 64, 64, 3), dtype=np.float32)
print(b.shape, b.size, b.nbytes)
```

▶ OUTPUT

```
(32, 64, 64, 3) 393216 1572864
```

Shape (32, 64, 64, 3). Size = 32 x 64 x 64 x 3 = 393,216. float32 is 4 bytes, so nbytes = 1,572,864.

Q8 THEORY

(i) An array stores one fixed type in one contiguous block and runs operations as compiled C loops; a list stores pointers to full objects with per-item type information and needs Python loops. **(ii)** np.arange(1, 10, 3) takes a step and excludes the stop, giving [1 4 7]; np.linspace(1, 10, 4) takes a count and includes the stop, giving [1. 4. 7. 10.]. **(iii)** A slice is a view sharing memory with the original, so modifying it changes the original; .copy() makes an independent copy.